

2025/2026

Acknowledgement

We would like to express our sincere gratitude to everyone who supported us throughout the development of this graduation project, **SCAN_GO – Smart Supermarket Checkout System**.

First and foremost, we extend our deepest appreciation to **Dr. Mohamed Kamal** for his invaluable guidance, continuous support, and insightful feedback, which played a crucial role in shaping this project.

We would also like to thank **Eng. Doria Gamal** for her assistance, encouragement, and technical support throughout the different phases of the project.

We are also grateful for the collaborative efforts of our team members, whose dedication and teamwork were essential in successfully completing this project. Special thanks to all individuals who provided constructive feedback and valuable suggestions that helped improve the quality of our work.

We also acknowledge the technological resources and tools that facilitated the development and implementation of this system.

Our appreciation also goes to the faculty members of the Computer Science Department at Misr Higher Institute for Commerce & Computers for providing us with the knowledge and resources necessary to complete this work.

Finally, we would like to thank our families and friends for their constant support, patience, and motivation during this journey.

Contents

Acknowledgement	2
ABSTRACT	5
Chapter One	6
1.1 Background and Context	7

1.2 Problem Statement	8
1.2.1 Long Waiting Times at Checkout	10
1.2.2 Inefficiency of Manual Checkout Processes	10
1.2.3 Complications Associated with Cash Transactions	11
1.2.4 High Operational Costs	11
1.2.5 Lack of Real-Time Inventory and Sales Monitoring	11
1.2.6 Inventory Shrinkage and Security Issues	12
1.2.7 Poor Customer Experience	12
1.2.8 Limitations of Existing Smart Retail Solutions	12
1.2.9 Summary of the Problem	13
1.3 Aim of the Project	13
1.4 Project Objectives	15
1.4.1 Development of a Mobile-Based Self-Scanning System.....	15
1.4.2 Implementation of a Digital Payment System	16
1.4.3 Design and Development of a Backend System	16

1.4.4 Creation of an Administrative Dashboard 16

1.4.5 Implementation of QR Code-Based Verification 17

1.4.6 Integration of System Components 17

1.4.7 Ensuring System Scalability18

1.4.8 Secure and Efficient Inventory Management 18

1.5 Proposed System Overview 18

1.6 System Architecture Overview 20

1.6.1 Mobile Application Layer 21

1.6.2 Backend System Layer 21

1.6.3 Database Layer 22

1.6.4 Admin Dashboard Layer 23

1.6.5 Communication Layer (API Integration) 23

1.6.6 Security Considerations in Architecture 23

1.6.7 Scalability and Performance 24

1.6.8 Summary of System Architecture	24
---	----

ABSTRACT

This project presents SCAN_GO, a smart basket-based checkout system designed to eliminate traditional cashier-based processes and improve the overall shopping experience. The system is inspired by cashier-less retail concepts such as Amazon Go but is specifically designed as a cost-effective and scalable solution suitable for developing environments. SCAN_GO operates by integrating a tablet, a microcontroller, a barcode scanner, and a weight sensor directly onto the shopping basket, allowing customers to scan items while automatically validating transactions through real-time weight verification. Unlike traditional systems that rely on manual checkout procedures, this approach effectively eliminates waiting times and enhances operational efficiency. The system consists of three main components: an Android application for customer interaction, a centralized backend API for handling business logic and data processing, and an administrative dashboard for real-time monitoring of sales and inventory. The proposed solution addresses several key challenges in conventional retail environments, including long checkout queues, inefficiencies in manual processing, and retail loss. Additionally, SCAN_GO incorporates a QR-code-based exit verification gate to ensure secure, accurate transactions, improved customer satisfaction, and enhanced inventory control. Overall, SCAN_GO provides a practical, highly scalable approach to modernizing retail operations.

CHAPTER 1:

(Introduction)

1.1 Background and Context

In recent decades, the retail industry has experienced a rapid transformation driven by technological advancements and evolving consumer expectations. Traditional shopping experiences, which once relied heavily on manual interactions between customers and cashiers, are gradually being replaced by automated and digital solutions. This transformation is not only a response to technological innovation but also a necessity to remain competitive in an increasingly fast-paced and customercentric market.

Supermarkets, in particular, represent a critical component of daily life, serving a wide range of customers with diverse needs. However, despite technological progress in other sectors, many supermarkets still depend on outdated checkout systems. These systems are characterized by long queues, manual barcode scanning, and reliance on physical cash transactions. Such inefficiencies result in reduced customer satisfaction and increased operational costs.

With the rise of smartphones, cloud computing, and digital payment systems, new opportunities have emerged to revolutionize the retail experience. Modern consumers demand speed, convenience, and flexibility, expecting seamless interactions without unnecessary delays. This shift in expectations has encouraged the development of innovative retail concepts such as cashierless stores.

One of the most notable examples is Amazon Go, which introduced a fully automated shopping experience using advanced technologies such as computer vision, machine learning, and sensor fusion. While this model represents a significant leap forward, it is associated with high implementation costs due to its reliance on complex infrastructure, including numerous cameras and sensors installed throughout the store.

In contrast, many developing regions face financial and infrastructural limitations that make such solutions impractical. Therefore, there is a strong need for alternative systems that can deliver similar benefits while remaining affordable and scalable.

The SCAN_GO system is proposed as a practical solution to bridge this gap. By utilizing an embedded smart shopping basket equipped with an Android tablet, an ESP32 microcontroller, a barcode scanner, and a weight sensor, the system enables customers to seamlessly scan products and complete secure digital payments. This localized hardware approach eliminates the need for expensive store-wide infrastructure while maintaining the core advantages of cashier-less shopping.

1.2. Problem Statement

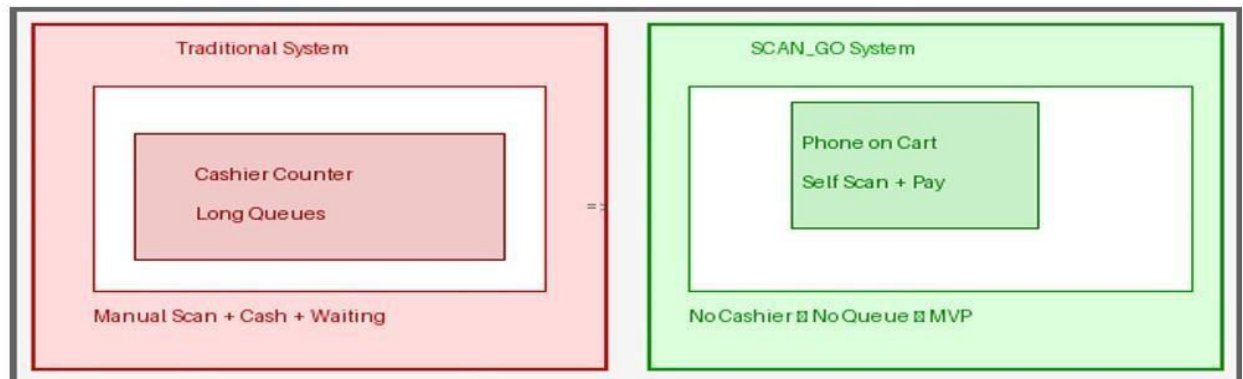


Figure 1.1 Comparison between traditional checkout and SCAN_GO system

Despite the rapid advancement in technology and the increasing adoption of digital solutions across various industries, traditional supermarket systems still rely heavily

on outdated checkout mechanisms. These systems, although functional, present several limitations that negatively impact both operational efficiency and customer satisfaction.

Supermarkets are designed to handle a large number of customers daily, especially during peak hours such as weekends and holidays. However, the traditional checkout process becomes a bottleneck in the overall shopping experience. Customers are required to wait in long queues, interact with cashiers, and go through repetitive manual procedures before completing their purchases. This process is not only time-consuming but also inefficient in handling high customer volumes.

Moreover, the reliance on human intervention introduces variability and inconsistency in service quality. The performance of the checkout process depends largely on the speed and accuracy of cashiers, which can vary from one employee to another. This inconsistency can lead to delays, errors in billing, and overall dissatisfaction among customers.

In addition, traditional systems are not well-equipped to meet the expectations of modern consumers who demand speed, convenience, and seamless experiences. With the widespread use of smartphones and digital payment solutions, customers increasingly prefer self-service options that allow them to complete transactions quickly and independently.

Another critical issue is the lack of real-time data accessibility. In conventional systems, store managers often rely on delayed reports to monitor sales and inventory levels. This delay in information flow affects decision-making processes, leading to inefficient stock management, overstocking, or stock shortages.

Furthermore, the dependency on cash transactions introduces multiple operational challenges.

Handling physical cash requires additional security measures, increases

the risk of human error, and slows down the transaction process. It also creates inconvenience for customers who prefer digital payment methods.

Security is also a major concern in traditional retail environments. Without an automated, crossreferenced verification system, supermarkets are highly vulnerable to inventory shrinkage caused by theft or unpaid items. This results in significant financial losses and reduced profitability.

To better understand the challenges faced by traditional supermarket systems, the problem can be categorized into several key areas:

1.2.1. Long Waiting Times at Checkout

One of the most visible and frustrating problems in traditional supermarkets is the extensive waiting time at checkout counters. During peak hours, customers often spend significantly more time waiting in lines than actually selecting their items.

This bottleneck arises from several systemic limitations, including a restricted number of operational checkout lanes, slow manual processing of individual items, and high customer volumes. Long queues drastically reduce customer satisfaction and degrade the overall retail experience.

In many instances, excessive delays lead to cart abandonment, resulting in direct sales losses for the store. Furthermore, checkout congestion creates physical crowding near the store exits, disrupting the overall traffic flow within the supermarket. By shifting the checkout process directly to the smart basket, this localized approach completely decentralizes the transaction point and eliminates the need for physical queues.

1.2.2. Inefficiency of Manual Checkout Processes

The traditional checkout workflow relies entirely on human intervention to manually scan barcodes, verify prices, process payments, and print physical receipts. This repetitive and labor-intensive routine inherently acts as an operational bottleneck.

Heavy reliance on manual handling escalates the probability of human errors, including pricing discrepancies, billing mistakes, or missed items. Furthermore, these sequential manual procedures cap the maximum throughput of the store, limiting its capacity to handle large customer volumes efficiently and driving up labor costs. Shifting this workflow to an automated, decentralized system via the smart basket allows for parallelized transaction processing, drastically boosting overall retail efficiency.

1.2.3. Complications Associated with Cash Transactions

Physical cash transactions introduce considerable friction and operational delays into the retail checkout workflow. Cashiers must manually count currency, calculate and dispense change, and verify banknote authenticity, which prolongs processing time per customer.

Furthermore, cash handling escalates the risk of human accounting errors, bookkeeping discrepancies, and security vulnerabilities such as internal theft or counterfeit fraud. Managing physical tender also imposes heavy administrative overhead on store management, including daily reconciliation, secure physical storage, and armored transport deposits.

From the consumer perspective, cash payments lack flexibility and convenience, particularly when managing change. By integrating direct digital payment gateways into the smart basket's internal software, the system eliminates cash-related overhead, streamlines financial auditing, and secures the transaction lifecycle.

1.2.4. High Operational Costs

Sustaining a conventional storefront architecture demands substantial financial expenditure to recruit, train, and manage a large cohort of cashiers. These fixed and variable labor costs—encompassing salaries, onboarding resources, and supervisory overhead—substantially deflate net profit margins.

Furthermore, under the traditional retail model, scaling operations to meet fluctuating consumer demands exhibits a linear dependency; spike periods require opening additional physical lanes, thereby compounding operational expenses. This resource-intensive framework limits long-term fiscal sustainability, especially for enterprises striving to achieve lean operations and cost optimization. By decentralizing the point of sale via the smart basket, the store can seamlessly handle high throughput without a proportional increase in personnel overhead.

1.2.5. Lack of Real-Time Inventory and Sales Monitoring

Conventional retail architectures fail to deliver instantaneous visibility into live store operations, forcing management to rely on delayed, periodic batch reports to evaluate sales velocity and inventory levels. This chronic latency in data synchronization severely impedes proactive decisionmaking, culminating in critical operational friction such as localized stock shortages, lost sales opportunities, or excessive overstocking.

Without access to highly granular, real-time transaction streams, maintaining an optimal inventory equilibrium becomes logistically unfeasible. By leveraging a network of smart baskets that instantly push telemetry data—such as immediate barcode validation and item subtractions—to a centralized cloud backend, the system bridges this data gap. This enables automated, real-time inventory reconciliation and predictive stock replenishment alerts.

1.2.6. Inventory Shrinkage and Security Issues

Inventory shrinkage represents a critical financial vulnerability within traditional retail environments, driven by systemic theft, product damage, and administrative discrepancies. Conventional storefront frameworks lack an automated, cross-referenced mechanism to verify that every item in a customer's physical possession has been accurately scanned and processed prior to exit.

This absence of real-time point-of-sale control significantly escalates retail loss and dampens net profitability. Furthermore, relying on manual surveillance to monitor high customer volumes is structurally inefficient and highly prone to human oversight. By embedding an automated weightverification engine—driven by the

\$HX711\$ load cell module—directly into the smart basket, the system continuously audits cart weight against scanned barcode metadata. This localized hardware validation, paired with a secure QR-code-

controlled exit gate, constructs an objective, tamper-resistant security layer that mitigates theft without operational friction.

1.2.7. Poor Customer Experience

All the previously mentioned bottlenecks collectively degrade the overall customer experience. Chronic checkout delays, outdated manual scanning, and rigid payment workflows foster consumer frustration and dissatisfaction. In today's highly competitive retail market, customer experience is a primary determinant of business retention and brand loyalty. Supermarkets that fail to meet modern convenience expectations risk losing substantial market share to competitors leveraging automated infrastructure. By decentralizing the checkout process through an interactive, basket-integrated interface, the system transforms a high-friction chore into a seamless, autonomous customer journey.

1.2.8. Limitations of Existing Smart Retail Solutions

Although pioneering computer-vision-based solutions like Amazon Go have validated the feasibility of cashier-less environments, their ubiquitous deployment remains constrained by prohibitive capital expenditure and intricate infrastructural prerequisites. These high-end ecosystems rely heavily on dense ceiling-mounted camera grids, distributed weight-sensing shelves, and continuous sensor fusion algorithms.

Consequently, the localized computing overhead and high maintenance costs make them economically unviable and technically impractical for small-to-medium supermarkets, particularly within developing economies. This creates an immediate market gap for a localized, edge-computed alternative. There is a critical need for an affordable, infrastructure-light solution that architecturewise shifts the intelligence from the building itself to the individual shopping vessel, achieving frictionless checkout without structural store modifications.

1.2.9. Summary of the Problem

In summary, conventional retail ecosystems are constrained by a network of interconnected operational challenges, predominantly characterized by throughput inefficiencies, escalating labor overhead, compromised customer satisfaction, and a critical deficit in real-time data synchronization. These systemic friction points underscore the urgent necessity for an innovative, infrastructure-light solution capable of modernizing store operations while maintaining fiscal and scalable viability. The proposed

SCAN_GO system directly addresses these multi-dimensional vulnerabilities, introducing a decentralized, sensor-integrated shopping basket that serves as a cost-effective, high-efficiency alternative to traditional checkout frameworks.

1.3. Aim of the Project

The primary objective of this project is to design, develop, and evaluate the SCAN_GO system—an infrastructure-light, decentralized smart supermarket checkout platform engineered to eliminate traditional checkout bottlenecks and replace them with an autonomous, user-centric, and hardware-integrated framework. The project aims to transition conventional retail operations into a digitized self-service experience by shifting the computing intelligence directly onto the individual shopping basket. By combining an Android tablet interface, an ESP32 microcontroller, a localized barcode scanner, and an \$HX711\$-driven load cell weight verification module, the system aims to automate item auditing and digital payment processing, thereby minimizing reliance on human intervention while preserving strict inventory security.

Furthermore, this project is fundamentally designed with cost-effectiveness and scalable deployment in mind. Unlike capital-intensive computer-vision frameworks, SCAN_GO focuses on utilizing accessible edge devices to deliver high-tier automation, making it structurally and financially viable for supermarkets in developing regions with infrastructure constraints. On an operational level, the system aims to optimize store resource allocation; automating data ingestion via edge hardware reduces administrative staff workload, allows human capital to redirect toward value-added tasks, and drives down overhead costs. For the consumer, the system targets a completely frictionless journey by enabling concurrent item scanning, real-time telemetry of total expenditure, and instantaneous digital transaction processing. Finally, the project aims to empower store management through real-time telemetry of sales and inventory dynamics to enable predictive replenishment, while deploying a secure, QR-code-activated exit gate to mitigate inventory shrinkage and validate transactional integrity.

1.4. Project Objectives



Figure 1.2 SCAN_GO – Smartphone mounted on shopping cart

To realize the overarching aim of the SCAN_GO platform, a set of strategic, measurable, and engineering-centric objectives has been defined. These objectives serve as the technical milestones guiding the architecture design, hardware-software integration, and empirical evaluation of the system.

1.4.1. Development of a Mobile-Based Self-Scanning System

A foundational objective of this project is to develop a high-performance Android application deployed directly onto a tablet interface mounted on the smart shopping cart. The user interface is engineered to provide an intuitive, seamless human-machine interaction, allowing customers to autonomously manage their shopping lifecycle.

The scanning module interfaces natively with localized hardware scanners to achieve rapid, lowlatency barcode ingestion. This sub-system guarantees high-precision item identification, immediately appending verified products to the user's localized digital cart. To enhance transparency and shopping flow, the application architecture executes real-time data binding to render dynamic telemetry, including precise unit pricing, localized item quantities, cumulative cart subtotals, and instantaneous promotional data updates.

1.4.2. Implementation of a Digital Payment System

Another core objective involves embedding a secure, low-latency digital payment architecture directly within the application layer. The system is designed to facilitate instantaneous point-of-sale transactions through various electronic payment methodologies, including tokenized credit/debit cards, decentralized mobile wallets, and secure online payment gateways.

The transactional pipeline prioritizes computational efficiency and cryptographic security to minimize checkout latency. To preserve data integrity and comply with financial security standards, the subsystem implements advanced security protocols, including End-to-End Encryption Secure Sockets Layer (SSL/TLS) communication channels, and tokenization. These operational safeguards protect sensitive user credentials, prevent unauthorized network access, and mitigate transaction-level fraud.

1.4.3. Design and Development of a Backend System

A critical objective of this project is to architect and deploy a robust, scalable centralized backend system designed to govern the platform's core business logic, real-time data processing, and relational storage. The backend engineering focuses on constructing a unified ecosystem that orchestrates seamless, bidirectional communication between the basket-integrated Android interface, the ESP32 edge hardware, and the centralized administrator dashboard via structured Application Programming Interfaces.

The backend infrastructure is engineered to optimize throughput and minimize latency when managing highly concurrent data streams, including cryptographic user credentials, extensive product metadata databases, and immutable financial

transaction records. To achieve long-term scalability and operational reliability, the system architecture decouples data ingestion from relational query processing, utilizing optimal database indexing and secure communication protocols to prevent race conditions and ensure strict data synchronization across all system nodes.

1.4.4. Creation of an Administrative Dashboard

A critical objective of this project is to design and implement a web-based administrative dashboard that serves as the centralized command-and-control center for retail management. The dashboard's

architecture focuses on consolidating high-velocity data streams pushed from the backend, providing store operators with real-time visibility into distributed checkout analytics, inventory fluctuations, and concurrent customer transaction cycles.

The front-end interface is engineered around interactive data visualization frameworks, incorporating responsive analytical charts, multi-dimensional reporting tools, and low-latency anomaly alerts. This design empowers management with a data-driven decision-making environment to monitor localized store performance and track hardware fleet activity. To ensure enterprise-grade security and operational integrity, the platform integrates Role-Based Access Control protocols, restricting system visibility and configuration rights strictly to authenticated and authorized personnel.

1.4.5. Implementation of QR Code-Based Verification

To preserve transactional integrity and enforce strict egress security, the system architecture incorporates a dynamic cryptographic QR code generation engine within the application layer. Upon the execution of a successful financial transaction, the backend database generates an immutable digital receipt encrypted into a localized QR code token.

This token serves as a tamper-resistant proof of purchase containing structured cart metadata. At the storefront perimeter, a dedicated hardware-integrated reader scans the generated QR code to crossreference customer transaction data with the cloud registry. This automated verification mechanism triggers the physical gate actuation, eliminating unauthorized product removal and effectively mitigating inventory shrinkage. By replacing subjective manual exit checks with objective hardware access control, this subsystem constructs a reliable, high-throughput security perimeter that protects both retail assets and consumers.

1.4.6. Integration of System Components

A compounding objective of this project is to orchestrate seamless end-to-end integration and interoperability across all distributed system components. The architecture is engineered to transform the standalone layers—the basket-integrated Android interface, the ESP32 embedded edge hardware, the centralized backend framework, the cloud database, and the web-based administrative dashboard—into a unified, cohesive retail automation ecosystem.

This objective necessitates the rigorous design and implementation of lightweight Application Programming Interfaces (\$APIs\$) and optimized communication protocols. By establishing secure, bidirectional JSON-payload data pathways the system architecture ensures low-latency data synchronization, eliminates race conditions during concurrent cart updates, and guarantees fluid telemetry flow from the physical shopping vessel to the administrative dashboard.

1.4.7. Ensuring System Scalability

The system should be designed in a way that allows it to scale efficiently as the number of users and transactions increases. This includes the ability to handle multiple stores, large product databases, and high traffic volumes.

Scalability ensures that the system can grow with the business and adapt to future requirements.

1.4.8. Secure and Efficient Inventory Management

The system should provide accurate and real-time data on inventory levels, enabling better stock management. This helps prevent issues such as overstocking or stock shortages.

Efficient inventory management contributes to improved profitability and customer satisfaction.

Security is a critical aspect of the system. Measures must be implemented to protect user data, secure transactions, and prevent unauthorized access.

The system should also be reliable, ensuring consistent performance without failures or downtime.

1.5. Proposed System Overview

The SCAN_GO platform introduces an innovative, decentralizing architecture engineered to overcome the throughput bottlenecks, high operational overhead, and infrastructure constraints inherent in traditional retail checkout models. Distinct from high-cost, computer-vision frameworks that require pervasive environmental modifications, SCAN_GO shifts the system's computational intelligence directly onto the individual shopping vessel. By utilizing an edge-computing tablet mounted on the

shopping cart interfaced with localized sensors, the platform achieves autonomous checkout capability with high fiscal and scalable viability.

- **Initialization and Edge Data Ingestion:** The shopping journey begins at a cart integrated with an Android-based terminal coupled to an ESP32 microcontroller network. As items are selected, the customer utilizes the localized barcode scanning

module to execute rapid serialization. The application layer performs asynchronous API calls to the centralized backend database to instantly retrieve and bind product metadata, including item nomenclature, unitary cost, and localized stock availability.

- **Real -Time Data Binding & Audit Trails:** Concurrently, each validated product updates a local digital cart instance. The front-end user interface renders real-time telemetry of cumulative expenditure, itemized sub-totals, and specific quantities. This continuous data visibility eliminates transactional ambiguity, empowering consumers with precise financial control before approaching the storefront perimeter.

- **Decentralized Checkout Execution:** A core advantage of this architecture is the distribution of transaction processing throughout the shopping temporal sequence, eliminating the traditional congestion points at centralized checkout lines. Upon cart completion, the payment phase is executed natively within the application layer via an authenticated, encrypted digital payment pipeline. Successful transaction resolution triggers the backend server to generate an immutable, tamper-resistant cryptographic QR code token representing the digital receipt metadata.

- **Access Control Verification & Analytics:** At the storefront egress point, a hardware-integrated optical scanner evaluates the cryptographic QR token against the cloud registry. Validated receipts automatically actuate the physical security gate, mitigating inventory shrinkage while preserving high-throughput exit dynamics.

Simultaneously, all transaction telemetry is ingested into the web-based administrative dashboard, equipping store management with real-time operational analytics for predictive replenishment and data-driven inventory control.

1.6. System Architecture Overview

The architecture of the SCAN_GO platform is engineered to ensure high throughput, horizontal scalability, and seamless interoperability across heterogeneous nodes. The system implements a decoupled, centralized-broker architecture model, wherein a high-performance backend server acts as the central orchestration layer to manage asynchronous data processing, state synchronization, and peripheral coordination.

The structural design is organized into a multi-tiered, layered architecture. This decoupling enforces strict modularity and separation of concerns (\$SoC\$), ensuring that individual system components—ranging from embedded hardware abstraction layers to cloud-hosted databases—can be developed, debugged, and scaled independently. This modular framework minimizes systemic dependencies, reducing maintenance overhead and providing a highly adaptable foundation for future enterprise retail integrations.

1.6.1. Mobile Application Layer

The client-side application layer acts as the primary human-machine interface for the customer, serving as the central orchestration point for user-system interaction. Deployed directly on the basket-integrated Android terminal, this layer is structurally responsible for executing localized edge interactions, including synchronous barcode data capture, stateful cart memory management, and secure cryptographic payment processing.

The application architecture utilizes reactive data-binding frameworks to deliver deterministic, lowlatency visual feedback, ensuring immediate interface state updates following any peripheral hardware trigger.

A foundational sub-module within this layer is the barcode ingestion engine. Interfacing natively with the cart's hardware scanner peripheral via localized

communication APIs, the subsystem translates raw optical captures into structured numerical payloads for backend validation. This pipeline is optimized for high-precision, sub-second decoding to eliminate transactional friction.

Structurally, the application layer encapsulates state management routines that enable users to execute the following core operations:

Dynamic Inventory Auditing: Instantiate and render asynchronous views of the local digital cart contents.

Stateful Quantity Mutation: Modify item quantities natively with automatic local verification.

Real-Time Expenditure Telemetry: Stream continuous, real-time computations of the cumulative cart cost, including discounts and tax parameters.

Transactional Pipeline Egress: Formally compile cart metadata and securely handover session control to the integrated digital payment gateway.

1.6.2. Backend System Layer

The backend system is the core component that manages all operations within the SCAN_GO system. It handles business logic, processes requests, and ensures communication between different components.

The backend is responsible for:

- Managing product data
- Processing transactions
- Handling payment verification
- Generating QR codes
- Storing user activity

It acts as a bridge between the mobile application and the database, ensuring that all data is processed efficiently and securely.

The backend must be designed to handle multiple concurrent users without performance degradation. This requires efficient data handling and optimized APIs.

1.6.3. Database Layer

The database layer is responsible for storing and managing all system data. This includes product information, transaction records, user data, and inventory details.

The database must be structured in a way that allows fast data retrieval and efficient updates. It should also ensure data consistency and integrity.

Key data stored in the database includes:

- Product catalog
- Pricing information
- Transaction history
- Inventory levels

1.6.4. Admin Dashboard Layer

The admin dashboard provides store managers with a comprehensive view of store operations. It allows them to monitor sales, track inventory, and analyze customer behavior in real time.

The dashboard includes various features such as:

- Sales reports
- Inventory tracking

- Transaction monitoring
- Data visualization

This layer plays a crucial role in decision-making, enabling managers to respond quickly to changes in demand and operational conditions.

1.6.5. Communication Layer (API Integration)

Communication between system components is handled through APIs (Application Programming Interfaces). The backend exposes endpoints that allow the mobile application and admin dashboard to interact with the system.

The communication is typically carried out over secure protocols such as HTTPS to ensure data protection. APIs must be well-designed to ensure reliability and performance.

1.6.6. Security Considerations in Architecture

Security is a fundamental aspect of the system architecture. Measures must be implemented to protect user data and ensure safe transactions.

These measures include:

- Data encryption
- Secure authentication
- Access control mechanisms
- Protection against unauthorized access

Additionally, the QR code verification system adds an extra layer of security by ensuring that only paid items leave the store.

1.6.7. Scalability and Performance

The system architecture is designed to support scalability. As the number of users increases, the system should be able to handle higher loads without affecting performance.

This can be achieved through:

- Efficient backend design
- Optimized database queries
- Scalable cloud infrastructure

Scalability ensures that the system remains effective as the business grows.

1.6.8. Summary of System Architecture

In summary, the SCAN_GO system architecture is designed to provide a robust, scalable, and efficient solution for smart retail operations.

The integration of mobile technology, backend processing, and real-time data management creates a unified system that enhances both customer experience and operational efficiency.

CHAPTER 2

Literature Review

This chapter presents a comprehensive review of existing research, academic studies, and commercial implementations in the domain of smart retail, cashier-less checkout systems, and self-scanning technologies. The review aims to identify the current state of the art, analyze the strengths and limitations of prior approaches, and establish the research gap that the SCAN_GO project addresses. By examining systems from North America, Europe, and Asia, as well as relevant academic literature, this chapter provides the theoretical and practical foundation upon which SCAN_GO is designed and developed.

2.1 Introduction to Smart Retail and Cashier-less Technology

The concept of smart retail has emerged as one of the most transformative forces reshaping the global retail industry. Driven by rapid advances in computer vision, artificial intelligence (AI), the Internet of Things (IoT), and mobile computing, smart retail systems aim to create seamless, efficient, and personalized shopping experiences that reduce or eliminate reliance on traditional cashier-based workflows (Pantano et al., 2018).

Traditional supermarket checkout processes have remained largely unchanged for decades. Customers gather items, proceed to a checkout lane, and wait for a cashier to scan each product individually before completing payment. This model is increasingly inadequate in the face of growing consumer expectations for speed, convenience, and digital integration (Grewal et al., 2017).

The literature on smart retail can be broadly categorized into three streams: (1) infrastructure-heavy autonomous systems relying on overhead cameras and deep learning; (2) RFID-based item tracking systems that attach electronic tags to individual products; and (3) basket-integrated self-scanning systems where customers scan product barcodes using a hardware scanner mounted directly on the shopping basket. SCAN_GO belongs primarily to the third

category, leveraging an ESP32 microcontroller, a dedicated barcode scanner, a weight sensor, and an Android tablet mounted on the shopping basket to create a cost-effective solution without significant store-wide infrastructure investment.

2.2 Amazon Go — The Pioneer of Cashier-less Retail (USA)

Amazon Go is widely regarded as the most prominent and technically sophisticated cashierless retail system in existence (Demaitre, 2018). First opened to the public in Seattle, Washington in January 2018, Amazon Go stores use a combination of computer vision, deep learning, and sensor fusion to enable a fully automated shopping experience under the brand's slogan: 'Just Walk Out Technology'.

2.2.1 System Architecture and Technology

The Amazon Go system relies on a dense array of overhead cameras mounted throughout the store, capable of tracking customer movements and identifying which products are picked from or returned to shelves (Florentine, 2018). This tracking is supported by shelf-mounted weight sensors that detect subtle changes when items are removed, providing an additional layer of confirmation

through sensor fusion. Computer vision algorithms, trained on massive datasets using deep convolutional neural networks, associate customers with a virtual cart throughout their visit.

2.2.2 Strengths

The principal strength of Amazon Go is the completeness of its automation — no scanning, no queuing, no manual payment initiation (Demaitre, 2018). The high accuracy of AI tracking minimizes billing errors and inventory discrepancies, and the system demonstrated for the first time at commercial scale that fully autonomous retail is technically feasible.

2.2.3 Limitations and Relevance to SCAN_GO

Despite its achievements, Amazon Go suffers from critical scalability and cost limitations. The installation cost for a single store has been estimated at several million US dollars, requiring hundreds of cameras, proprietary sensor hardware, and significant computational infrastructure (Hamilton, 2019). It also requires major store layout modifications, making it impractical for retrofitting existing supermarkets. SCAN_GO directly addresses this barrier by replacing store-wide overhead infrastructure with a smart basket-integrated barcode scanner and weight sensor, eliminating the need for any structural store modifications.

Key Takeaway from Amazon Go:

While Amazon Go proves the viability of cashier-less retail at a premium cost, SCAN_GO achieves a comparable outcome — eliminating checkout queues and enabling digital payment — through a basket-first, infrastructure-light approach that is orders of magnitude cheaper to implement.

2.3 Standard Cognition — AI-Powered Autonomous Checkout (USA)

Standard Cognition is a San Francisco-based startup founded in 2017, positioning itself as a provider of autonomous checkout technology deployable in existing retail stores without requiring a complete overhaul (Standard Cognition, 2021). The company uses ceiling-mounted AI cameras and computer vision software to track customers and their product selections.

2.3.1 System Description and Closure

The company raised over USD 150 million in venture funding and partnered with retailers in North America and Japan. However, in January 2023, Standard Cognition announced it was shutting down, citing the extreme difficulty and cost of achieving reliable autonomous checkout

at commercial scale (Perez, 2023). This closure is a critical data point demonstrating that even well-funded vision-based systems face fundamental viability challenges.

2.3.2 Key Lessons for SCAN_GO

For SCAN_GO, this outcome reinforces the decision to rely on explicit hardware-integrated barcode scanning mounted on the shopping basket rather than passive computer vision tracking. By making product identification a deliberate, basket-level hardware action, SCAN_GO eliminates store-wide infrastructure investment while achieving cashier-less checkout.

2.4 AiFi — Scalable Autonomous Retail Platform (USA / Europe)

AiFi is a prominent player in autonomous retail, founded in 2018 in Santa Clara, California. Unlike Amazon Go, AiFi's platform is explicitly designed for multi-retailer deployment as a white-label solution (AiFi, 2022). The system uses overhead cameras combined with deep learning for real-time shopper and product tracking.

2.4.1 Deployments

AiFi has deployed its technology in collaboration with various retail chains in the United States and Netherlands, including a partnership with OAK in Amsterdam and stadium concessions at State Farm Arena (AiFi, 2022). The system scales from small convenience stores to larger supermarket formats.

2.4.2 Comparison with SCAN_GO

AiFi represents an improvement over Amazon Go in business model flexibility, but underlying technology costs remain high. SCAN_GO requires no camera installation whatsoever; product identification is handled by a dedicated hardware barcode scanner embedded in each smart basket — inherently more scalable to budget-constrained environments such as Egyptian supermarkets.

2.5 European Smart Retail Systems

Europe has been a significant arena for smart retail development, with major grocery chains investing in approaches ranging from AI-powered autonomous

stores to mobile-app-integrated checkout systems. This section examines the most prominent European implementations.

2.5.1 Tesco GetGo — United Kingdom

Tesco, the UK's largest supermarket chain, launched its GetGo cashier-less format in London in 2021 in partnership with Trigo Vision, an Israeli startup specializing in frictionless retail (Butler, 2021). The store uses overhead cameras and deep learning computer vision to automatically track products customers pick up, charging them through the Tesco app.

Compared to SCAN_GO, Tesco GetGo requires dedicated ceiling camera installations and significant backend computing investment (Butler, 2021). It is also designed for small-format convenience stores, limiting scalability. SCAN_GO, using a basket-integrated scanner and weight sensor, requires no store-side hardware modifications beyond the smart baskets themselves.

2.5.2 Rewe Pick&Go — Germany

Rewe launched its Pick&Go autonomous checkout concept in Cologne in 2021, using overhead cameras and AI-based product recognition with QR-code customer authentication via the Rewe mobile app (Retail Technology Innovation Hub, 2022). Early pilot results indicated high customer satisfaction with average shopping trips under 15 minutes.

2.5.3 Carrefour Flash — France

Carrefour opened its first autonomous store, Carrefour Flash, in Paris in 2019 in collaboration with AiFi (Green Seed Group, 2019). The store demonstrates that major traditional supermarket chains can adapt to new technology paradigms, but has remained limited to small-format locations due to high per-store infrastructure costs — a limitation SCAN_GO directly avoids.

2.5.4 Summary of European Approaches

European smart retail initiatives share common characteristics: computer vision and AI for autonomous product identification, significant store infrastructure investment, primarily small-format deployment, and mobile app integration for authentication. While achieving genuine frictionless checkout, their cost and complexity limit deployment to well-funded retailers in developed markets. SCAN_GO draws inspiration from these customer experience goals while addressing their fundamental cost limitation.

2.6 Asian Smart Retail Systems

Asia, particularly China, Japan, and South Korea, has been at the forefront of smart retail innovation. Several Asian systems prioritize mobile-based solutions and QR code technology, aligning more closely with SCAN_GO's design philosophy than their Western counterparts.

2.6.1 Alibaba Freshippo (Hema) — China

Alibaba's Freshippo (Hema) represents one of the most ambitious smart retail implementations globally. Launched in 2016, it integrates online and offline shopping through the Alipay mobile app, combining RFID tags, barcode scanning via smartphones, and face recognition technology at checkout (Liu & Xu, 2020). Freshippo demonstrates that a barcode-scanning approach can scale to large supermarket formats.

However, Freshippo's reliance on face recognition raises significant privacy concerns and requires expensive biometric hardware (Liu & Xu, 2020). SCAN_GO replaces face recognition with QR code-based exit verification, achieving similar security outcomes without privacy implications or hardware costs beyond the smart basket unit itself.

2.6.2 JD.com X-Mart — China

JD.com launched its X-Mart concept stores in Beijing and Shanghai in 2018, featuring AI cameras, RFID-tagged products, and autonomous robotic restocking systems (JD.com, 2018). While technologically impressive, X-Mart represents a maximum-investment solution — SCAN_GO achieves cashier-less checkout using a basket-embedded barcode scanner and weight sensor, with no RFID tags, overhead cameras, or robots required.

2.6.3 BingoBox — China

BingoBox pioneered the fully unmanned, small-format convenience store, founded in 2016. Stores use RFID readers, QR code scanning for entry, and self-checkout kiosks with WeChat Pay/Alipay integration (TechNode, 2018).

BingoBox is perhaps the closest Chinese comparison for SCAN_GO because both rely on QR code-based verification and digital payment. However, BingoBox's RFID approach requires all products to be individually tagged — SCAN_GO uses standard barcodes already printed on products, read by the basket-integrated hardware scanner, eliminating product-level hardware modification entirely.

2.6.4 Smart Retail in South Korea

South Korea has been a regional leader in retail digitization. GS25 and Lotte have piloted cashier-less formats using IoT shelf sensors and QR code-based digital payment, reflecting the country's high smartphone penetration and digital payment adoption (Korea Retail Institute, 2021). Korea's

experience demonstrates that QR code and barcode-based checkout can achieve widespread adoption — evidence that SCAN_GO's foundational approach is commercially viable.

2.6.5 Seven-Eleven Japan — Cashierless Pilots

Seven-Eleven Japan has conducted extensive cashier-less pilots in partnership with AiFi and NEC, driven by Japan's acute retail labor shortage due to an aging population (NEC Corporation, 2020). Japanese pilots combine basket-level scanning, AI cameras, and IC card-based payment. Notably, Japan's experience highlights that basket-based checkout must be designed with accessibility in mind — directly informing SCAN_GO's interface design decisions prioritizing simplicity for all age groups.

2.7 Review of Academic Literature

Beyond commercial implementations, substantial academic research has investigated the technical challenges, human factors, and business implications of smart retail and cashier-less checkout systems.

2.7.1 Mobile Self-Scanning and Consumer Behavior

Research by Hossain et al. (2020) examined adoption of self-scanning applications in European supermarkets, finding that perceived ease of use, time savings, and privacy concerns were the primary factors influencing adoption intention. Customers using self-scanning systems reported significantly higher satisfaction with the checkout experience, primarily due to reduced waiting times.

A complementary study by Van der Heijden and Sorber (2019) found that providing clear visual feedback on cart contents and total cost during shopping significantly increases perceived usefulness and actual usage frequency — a finding directly incorporated into SCAN_GO's Android tablet interface through real-time cart display and running total calculation.

2.7.2 Barcode Scanning Accuracy and Performance

Müller and Richter (2021) evaluated barcode scanning accuracy under retail lighting conditions. Testing EAN-13 and UPC-A barcodes across fifteen scanner configurations, they found modern hardware barcode scanners achieved accuracy rates exceeding 98% under standard supermarket lighting, rising to

over 99.5% in well-lit environments — confirming SCAN_GO's technical viability for commercial deployment.

2.7.3 QR Code-Based Verification Systems

Chen et al. (2022) analyzed QR-based payment verification in Chinese digital payment systems. Time-limited QR codes with cryptographic signing mechanisms provided robust security against fraud while maintaining checkout verification under three seconds on average. SCAN_GO's exit verification mechanism draws directly on this research model.

2.7.4 Real-Time Inventory Management

Garaus and Wagner (2019) analyzed inventory management impact of smart checkout systems across fifteen supermarket pilots in Austria and Germany, finding real-time transaction recording reduced inventory discrepancies by an average of 34% compared to traditional batch updates. This directly supports SCAN_GO's inclusion of a real-time administrative dashboard as a core system component.

2.7.5 Digital Payment Adoption in Developing Markets

Ozili (2018) found that acceptance of digital payment solutions in African markets was strongly correlated with perceived security, ease of use, and alignment with existing payment habits. Egypt has experienced significant growth in digital payment adoption, with the Central Bank of Egypt reporting a 300% increase in mobile wallet transactions between 2019 and 2023 — validating SCAN_GO's payment integration strategy for the Egyptian market.

2.8 Traditional Self-Checkout Kiosks — A Transitional

Technology

Research by Collier et al. (2021) found that self-checkout kiosks reduce cashier labor costs by 40% on average while improving throughput by 25% during peak hours. However, they still require customers to checkout at a fixed terminal, meaning queues persist. The ECR Retail Loss Group (2020) further found kiosks were associated with shrinkage rates 2–3 times higher than cashier lanes, due to intentional and accidental scanning failures.

SCAN_GO addresses both limitations: by distributing scanning throughout the shopping journey via the basket-integrated hardware scanner and weight sensor, checkout queues are eliminated entirely; and the QR code exit verification gate ensures only customers who completed payment can exit — providing stronger anti-theft assurance than kiosk-only implementations.

2.9 Comprehensive Comparative Analysis

The following table provides a structured comparison of SCAN_GO against the principal smart retail systems reviewed in this chapter. Cost and scalability ratings are defined in the footnotes below the table.

System	Country	Core Technology	HW Cost (a)	Scalability (b)	Cashierless	Basket/App	SCAN_GO Advantage
--------	---------	-----------------	-------------	-----------------	-------------	------------	-------------------

Amazon Go	USA	Computer Vision + AI + Sensor Fusion	Very High	Low	✓ Yes	Partial	Much lower cost; no infra needed
Standard Cognition	USA	AI Camera Networks (Ceilingmounted)	Very High	Low	✓ Yes	XNo	Avoids heavy AI infra; still operational
AiFi	USA/EU	Autonomous AI Checkout (White-label)	High	Medium	✓ Yes	Partial	Open to any retailer; no proprietary lock-in
Tesco GetGo	UK	Computer Vision + Deep Learning	High	Low	✓ Yes	✓ Yes	No ceiling camera installation required
		(Trigo)					
Rewe Pick&Go	Germany	Overhead Cameras + AI Tracking	High	Low	✓ Yes	✓ Yes	No fixed infrastructure modifications
Carrefour Flash	France	Computer Vision Cameras (AiFi-powered)	High	Low	✓ Yes	✓ Yes	Cost-effective alternative approach
Alibaba Freshippo	China	RFID + Face Recognition + AI	High	Medium	✓ Yes	✓ Yes	No face recognition; preserves privacy
JD.com X-Mart	China	AI Cameras + RFID Tags + Autonomous Robots	Very High	Low	✓ Yes	Partial	No RFID tagging or robotic systems needed
BingoBox	China	RFID + QR Entry + Self-Checkout Kiosk	Medium	High	Partial	✓ Yes	Fully basketbased; no RFID product tagging
Smart Retail Korea	S. Korea	IoT Shelf Sensors + QR + Digital Payment	Medium	High	Partial	✓ Yes	Simpler deployment; lower per-store cost

SCAN_GO (Proposed)	Egypt	Basket Scanner + Weight Sensor + ESP32 + QR Exit Verify	Very Low	Very High	✓ Yes	✓ Yes	— Proposed System —
-----------------------	-------	---	-------------	-----------	-------	-------	------------------------

Table 2.1 — Comparative Analysis of Smart Retail and Cashier-less Checkout Systems

Table Notes & Definitions:

(a) Hardware Cost Scale: Very High = >\$1M per store installation | High = \$200K–

\$1M | Medium = \$20K–\$200K | Very Low = <\$5K (basket hardware per unit only, no store modifications)

(b) Scalability Scale: Very High = deployable across hundreds of stores with minimal incremental cost | High = moderate hardware per store | Low = requires store-specific installation (cameras, sensors)

(c) Cashier-less: ✓Yes = no cashier required at any point | Partial = reduced cashier role (e.g. kiosk assistance still needed) | XNo = cashier still required

Sources: Demaitre (2018); Standard Cognition (2021); AiFi (2022); Butler (2021); Retail Technology Innovation Hub (2022); Green Seed Group (2019); Liu & Xu (2020); JD.com (2018); TechNode (2018); Korea Retail Institute (2021).

The comparative analysis reveals that all high-profile cashier-less implementations rely on computer vision and overhead camera infrastructure, imposing high costs limiting applicability to well-funded retailers in developed markets. Systems most closely approaching SCAN_GO's cost profile — BingoBox and Korean IoT/QR systems — still rely on hardware modifications at the product or shelf level. SCAN_GO uniquely achieves cashier-less checkout by embedding intelligence directly into the shopping basket: a dedicated barcode scanner, an HX711-driven weight verification module, and an Android tablet interface — requiring no per-product tagging, no sensor installation in the store, and no camera systems.

2.10 Identification of the Research Gap

The literature review reveals a clear research gap: the absence of a cost-effective, basket-integrated, infrastructure-light cashier-less checkout solution suitable for deployment in supermarkets in developing and emerging market contexts (Ozili, 2018; Hossain et al., 2020).

Existing solutions fall into two categories. The first — autonomous computer vision systems — achieves genuine cashier-less checkout at prohibitive infrastructure costs. The second — kiosks and RFID systems — reduces cashier reliance but requires hardware investment and does not fully eliminate checkout queues.

Basket-integrated self-scanning systems have been explored in academic research (Van der Heijden & Sorber, 2019) and limited commercial deployments, but these lack integrated exit verification mechanisms and have not been designed for the economic constraints of developing market retail environments. SCAN_GO fills this gap with a comprehensive architecture — basket-embedded barcode scanning, real-time weight verification via the HX711 load cell, real-time cart management on the Android tablet, integrated digital payment, exit QR verification gate, and administrative inventory monitoring — that achieves full cashier-less capabilities without any store-side infrastructure modification beyond deploying the smart baskets.

2.11 Chapter Summary

This chapter has provided a comprehensive review of the existing landscape of smart retail and cashier-less checkout systems, examining implementations from the United States, United Kingdom, Germany, France, China, South Korea, and Japan, alongside relevant academic research on self-scanning, barcode accuracy, QR code verification, inventory management, and digital payment adoption.

The review demonstrates that while significant technological progress has been made, existing solutions are characterized by high implementation costs, dependency on custom infrastructure, and limited applicability to retail environments with constrained capital budgets. Traditional self-checkout kiosks fail to eliminate checkout queues and present unresolved theft prevention challenges.

Academic research confirms the technical viability and consumer acceptance of hardware barcode scanning as a checkout mechanism, and supports the QR

code verification approach adopted by SCAN_GO for exit security. Research on digital payment adoption in developing markets provides evidence that Egyptian consumers are increasingly prepared to engage with digital payment solutions.

Together, these findings establish a clear research gap and validate SCAN_GO's proposed basket-integrated approach as a novel and practically viable contribution. The following chapter will present the system design and architecture of SCAN_GO.

Chapter Three

System Analysis

3.1. Introduction

The transition from conceptual software requirements to a physical, production-ready technical implementation represents a critical developmental stage in the engineering lifecycle. This chapter delivers a comprehensive and multifaceted architectural analysis of the Scan_Go framework, which is fundamentally structured upon the contemporary concept of an automated Smart Basket Market environment. The analytical scope encompasses the baseline distributed architecture, the complex structural logic of multi-device hardware workflows, and the highly specific engineering quality standards required to convert conventional grocery checkout points into fully automated self-service nodes. By systematically dissecting both structural functional operations and systemic non-functional constraints, this chapter establishes an academic and technical blueprint designed to govern the interaction between cart-mounted computing tablets, electronic weight validation sub-systems, client mobile devices, and centralized backend server APIs.

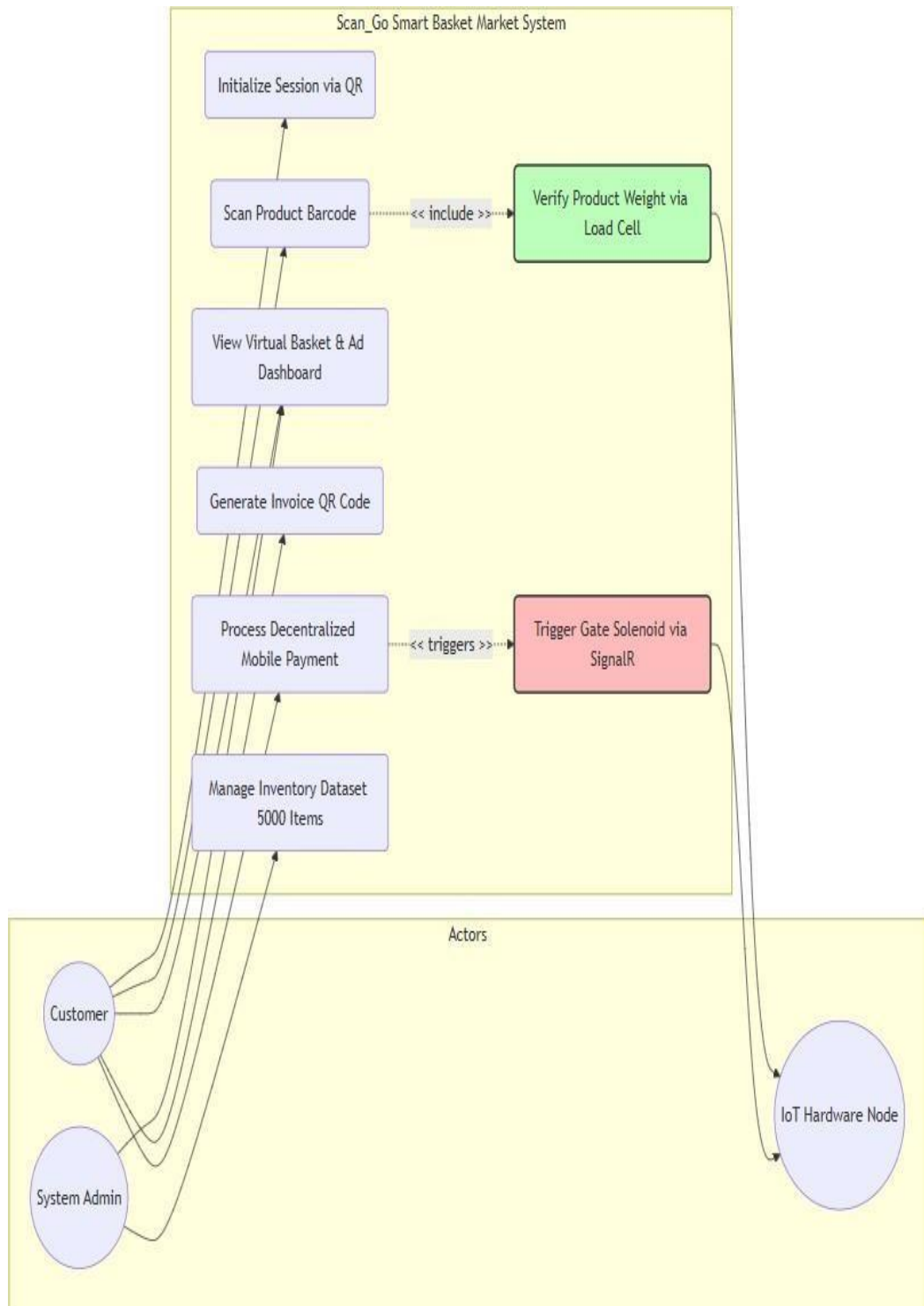
3.2. Overview

The Scan_Go platform integrates the operational principles of the Smart Basket Market paradigm, wherein technical design patterns focus entirely on optimizing and automating the standalone consumer shopping journey rather than restructuring the physical facility. Instead of employing cost-prohibitive overhead camera matrices or continuous RFID shelf configurations, the hardware layout leverages consumer baskets integrated with specialized low-cost Android tablet hardware, electronic weight validation sensors, and digital barcode modules. From an architectural perspective, the cross-device system utilizes a specialized client tablet application developed via the Flutter SDK for active grocery logging, a physical electronic load cell circuit connected via an HX711 amplifier to capture physical weight differentials, an isolated Flutter mobile application on the consumer device for financial settlement, and a centralized ASP.NET Core application server to maintain real-time data integrity and real-time cross-device communication. This multi-terminal deployment ensures that product processing, cryptographic identification, and validation loops occur directly within the decentralized cart boundary.

3.3. Functional Requirements

Functional requirements explicitly define the mandatory operational capabilities, processing protocols, and interface transactions that the Scan_Go platform must execute to sustain standard commercial operations. These software behaviors are partitioned into independent specialized functional modules that synchronize across active hardware terminals:

- **User Authentication Module:** Deploys secure identity verification mechanisms using ASP.NET Core Identity to bind client mobile profiles to independent cart computing terminals via dynamic QR code handshakes.
- **Dynamic Inventory Discovery:** Executes real-time database queries through database indexing configurations to fetch product names, synchronized pricing parameters, and exact item mass values upon barcode trigger events.
- **Real-time Weight Validation:** Processes continuous digital signal inputs from the physical basket load cell to compare real-world weight changes against expected server product masses before updating the virtual cart state.
- **Virtual Basket Logic:** Manages reactive software states to compile running mathematical totals, applicable local sales taxes, and commercial promotional modifications directly on the cart tablet presentation layer.
- **Automated Exit Control:** Operates a secure network protocol interface that issues physical opening signals via the SignalR framework to the boundary gate microcontroller only after transactional settlement confirmation.



3.4. Detailed Workflow

The operational journey inside the Scan_Go environment follows a rigid sequential workflow designed to eliminate manual cashier interventions while maintaining strict verification loops at each stage. The multi-device interaction cycle is executed through seven standardized sequential operational phases:

- The customer selects an available shopping basket integrated with a computing tablet interface and an electronic load cell weight sensor upon arriving at the retail floor.
- The customer uses their personal mobile device to scan the session initialization QR code displayed on the cart-mounted tablet terminal, which establishes a secure realtime WebSocket link between both user terminals.
- The customer scans the commercial barcode of a chosen product using the physical basket scanning module, which prompts the tablet application to pull the verified item specifications from the database server.
- The customer deposits the scanned item into the physical basket, forcing the underlying load cell circuit to transmit the weight increase to the tablet application for cross-reference validation.
- The system verifies the weight shift matches the product metadata; if verified, the virtual basket updates, and upon shopping completion, the customer triggers the termination command to generate an invoice QR code.
- The customer scans the generated invoice QR code using the Scan_Go application on their personal mobile terminal, allowing them to complete the payment transaction securely via isolated payment channels.
- The customer approaches the physical boundary gate where the server confirms payment verification and triggers the physical ESP8266 microcontroller to automatically unlock the solenoid gate mechanism.

3.5. Results Aggregation & Recommendation Dashboard

The data architecture layer features an analytical results aggregation engine designed to harvest real-time user selections during active shopping operations. This engine evaluates purchasing behaviors to compile valuable metrics that benefit both consumer operations and administrative stock oversight. For the consumer, the platform populates an algorithmic recommendation dashboard directly on the basket tablet screen, evaluating current item weight changes and product groups against historical shopping trends to display contextual ads. Simultaneously, all aggregated session logs are routed to an administrative management dashboard to keep inventory tracking completely automated.

3.6. Non-functional Requirements

Non-functional requirements specify the operational quality constraints, architectural thresholds, and structural boundaries that dictate global system performance. These metrics ensure the collective environment remains resilient under conditions of dense concurrent transactions.

3.6.1 Accuracy and Sensor Calibration

Accuracy is the primary critical factor governing the structural viability of the basket load cell system and database item mapping. The hardware system utilizes calibrated HX711 analog-to-digital amplifiers to ensure weight differentials on the shopping basket are tracked down to single-gram sensitivity. This prevents data exploitation by cross-matching real physical weight fluctuations with high-precision decimal datatypes managed within the .NET backend API.

3.6.2 Security

Systemic security is enforced using cryptographic validation models where all transactional API communications require JSON Web Tokens for execution. This security model blocks malicious network injections and ensures that hardware commands, such as gate unlocking triggers, originate exclusively from verified backend services. The server code is fully hardened against standard web vulnerabilities including Cross-Site Scripting.

3.6.3 Privacy

User data privacy is strictly protected by implementing data minimization workflows where sensitive consumer profile elements are heavily encrypted at rest in the database layers. The platform limits active session monitoring to historical item metrics required for recommendation calculations. Shoppers preserve the explicit right to disable analytics logging via the mobile application settings configuration.

3.6.4 Usability

The graphical layout structures generated via the Flutter SDK follow modern UI accessibility principles tailored for dynamic environments. Text visibility, click target boundaries, and contrast ratios are customized to support a wide demographic of shoppers interacting with cart hardware. The physical workflow is designed to minimize required screen inputs to reduce friction for new consumers.

3.6.5 Reliability

Platform availability is sustained via specialized offline synchronization states built directly into the basket software components. If local wireless network coverage drops during shopping, the cart tablet continues logging items and tracking weights locally via reactive client states. Upon network restoration, the system executes an automated synchronization handshake with the ASP.NET Core backend.

3.6.6 Performance

Computational performance is quantified by the end-to-end processing latency of database requests and sensor data verification cycles. The database architecture is highly indexed to process incoming barcode and weight validation routines within a maximum two-hundredmillisecond response window. This prevents interface lag and maintains a seamless transaction flow on the basket terminal.

3.7. Dataset Description

To confirm the computational stability and analytical correctness of the Scan_Go platform under heavy retail transactional loads, a comprehensive inventory dataset was designed. This structural repository supplies the extensive data schemas needed to validate index performance and sensor matching logic.

3.7.1 Data Volume and Scale

The database configuration contains exactly five thousand distinct product records, simulating an authentic mid-sized supermarket product ecosystem. This volume allows the developers to rigorously test search algorithm speeds, continuous relational data integrity, and complex concurrent query indexing rules within the centralized ASP.NET Core environment.

3.7.2 Data Source (Simulated Real-World Data)

The technical dataset represents a simulated inventory database engineered to match real grocery metrics while remaining highly controllable during software validation cycles. The product files are modeled on industrial retail parameters, using genuine GS1 GTIN-13 barcode configurations, standard market pricing variables adapted to the local market, and specific mass allocations (in grams) to support the weight sensor validation module.

3.7.3 Dataset Structure (Example Table)

The data table below showcases a structural example of the inventory schema, highlighting the integration of the expected product weight fields required by the smart basket verification loop:

Product ID	Barcode (GTIN)	Product Name	Category	Expected Weight (g)	Price (EGP)	Stock Qty
1001	6223000123456	Juhayna Full Cream Milk (1L)	Dairy	1030	38.00	150
1002	6221234567890	Molto XL Chocolate	Snacks	85	10.00	300
1003	6224000987654	Dasani Water (1.5L)	Beverages	1540	8.50	500
1004	6221000112233	Kellogg's Corn Flakes	Breakfast	375	120.00	45
1005	6225000445566	Ariel Gel (2.5L)	Detergents	2600	210.00	25

3.7.3 Data Integrity and Validation

Data integrity frameworks enforce specific transactional check constraints during the server database seeding routines. The API controllers block duplicate barcode strings and verify that all sensor parameters contain valid positive decimal configurations before database tables are approved for live runtime execution.

3.8. Component

The structural architecture of the Scan_Go ecosystem relies on a precise combination of industrial hardware components and scalable software frameworks to form a cohesive system:

- **Hardware Nodes:** Comprises Android tablets, consumer mobile phones, ESP8266 Wi-Fi microcontrollers, HX711 weight amplifiers, industrial load cells, 5V relay switches, and 12V solenoid door latches.
- **Software Environments:** Utilizes the Flutter SDK for native client interfaces, ASP.NET Core for building microservices, and Entity Framework Core as the ObjectRelational Mapper for transaction tracking.
- **Communication Infrastructure:** Employs highly optimized JSON-based RESTful APIs for database retrieval and full-duplex SignalR WebSockets for immediate boundary gate hardware automation.

3.9. System Constraints

The real-world implementation of the Smart Basket Market architecture operates under several hardware, electrical, and structural constraints that must be handled during commercial installation:

- **Wireless Network Dependency:** The cross-terminal communication protocols require stable and continuous local wireless network signal coverage across the complete retail floor area.
- **Electrical Power Constraints:** The mobile tablets and electronic load cells attached to shopping baskets require high-capacity power regulation units to guarantee continuous execution through store business hours.

- **Sensor Tare Calibration:** The weight validation sub-system demands routine digital tare adjustments to account for structural basket wearing and environmental debris aggregation.

3.10. Software Architecture Patterns

The software backend design implements the architectural Repository Pattern coupled with native Dependency Injection to isolate core database transactions from API controllers. This abstraction allows developers to change underlying data engines with minimal impact on business rules. The API depends on the Model-View-Controller pattern, while the presentation applications leverage the Business Logic Component pattern to handle cart states independently from UI view elements.

3.11. Hardware-Software Interface Protocol

The interface bridging high-level network applications to low-level physical automated gates operates via a specialized real-time communication design. Upon successful transaction processing, the server triggers a SignalR broadcast to the exit terminal NodeMCU ESP8266. The microcontroller operates a responsive, non-blocking monitoring loop that parses the cryptographic command payload. Upon successful signature validation, the internal GPIO pins transition to a HIGH state, toggling the relay switch and providing electrical power to retract the solenoid lock device within milliseconds.

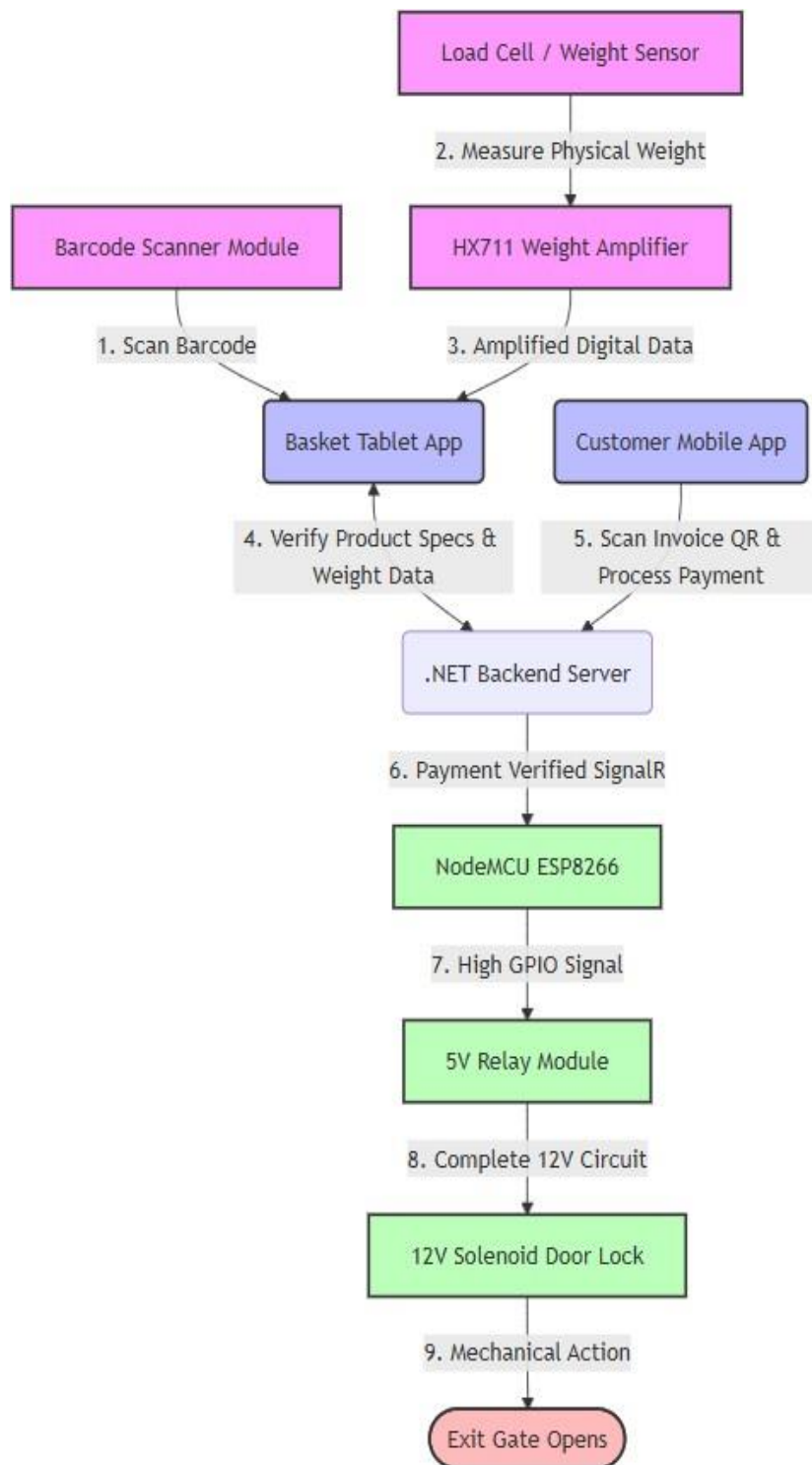
3.12. Evaluation & Comparative Analysis

To confirm the system improvements introduced by the automated Smart Basket Market model, a rigorous comparative analysis was performed against conventional retail pipelines. The evaluation tracks throughput capacity, verification safety, and transaction speed.

3.12.1 Traditional vs. Scan_Go Comparison

The comparative table below details the strategic functional upgrades realized by deploying cart-based sensor arrays versus manual counter operations:

Feature	Traditional Checkout	Scan_Go Smart Basket
User Interaction	Passive (Waiting in line)	Active (Self-scanning & verification)
Product Scanning	Manual by Cashier staff	Automated by Basket Tablet Scanner
Security Validation	Visual inspection by staff	Automated Load Cell Weight Verification
Payment Location	Centralized physical POS Counters	Decentralized (Personal Customer Mobile)
Human Intervention	High (Demands constant store staff)	Minimal (Autonomous validation loops)
Queue Management	High Congestion Risks at counters	Flow-based (No retail bottlenecking)



3.12.2. Key Performance Metrics

Platform efficiency is verified using standard industrial checkout metrics. Time saved per consumer interaction represents a core performance milestone; manual queue latency is completely bypassed, generating an average transaction processing time reduction of sixty to seventy-five percent per session. Concurrently, data validation testing shows that integrating automated load cell physical weight cross-referencing with backend database metadata fields keeps inventory discrepancy metrics below two percent, achieving high structural asset protection.

3.12.3. Preliminary Evaluation Summary

The primary performance analysis demonstrates that distributing product scanning actions to decentralized cart computing terminals and securing session metrics via physical weight data loops yields high asset tracking precision. This structural framework reduces corporate employee overhead expenses while maintaining strict real-time data validity across all active hardware nodes.

3.13. Summary

This chapter provided a comprehensive, technically dense analysis of the engineering principles governing the execution of the Scan_Go platform. By explicitly charting crossterminal functional interactions, data structures, weight sensor constraints, and real-time hardware interface rules, it defines the necessary parameters for the implementation phase. The balance between the ASP.NET Core ecosystem and responsive Flutter terminal applications delivers a highly secure, scalable, and commercially optimal alternative to traditional retail workflows.

Chapter 4:

System Design and Modeling

4.1. Introduction to System Architecture

The structural transition from conceptual requirements to a functional digital infrastructure requires comprehensive architectural modeling to define system behavior accurately. This chapter delineates the foundational blueprints of the Smart Basket Market platform, utilizing standardized modeling languages to illustrate complex interactions across diverse hardware and software boundaries. By isolating the automated self-service checkout processes from traditional retail workflows, the models prioritize decentralized product validation protocols executed directly within the shopping cart module. The core objective of this design is to establish a rigorous, highly available communication mechanism between client applications and central databases without compromising structural security.

Following strict engineering protocols, the subsequent sections partition the systemic framework into multiple isolated perspectives, encompassing functional use cases, chronologically ordered sequence flows, dynamic operational activities, static class structures, and persistent data entities. These multi-dimensional abstractions serve as the technical foundation required to guide the physical deployment of sensor hardware and the programmatic integration of mobile interfaces. Consequently, the depicted models validate the procedural logic responsible for translating physical weight differentials into cryptographic transactional settlements.

4.2. Use Case Model

The Use Case Diagram defines the operational scope of the framework by specifying the authorized actors and their corresponding privileges within the isolated digital ecosystem. This abstraction acts as a boundary definition, strictly outlining which entities may trigger functional processing routines during an active retail session. Within the Smart Basket Market architecture, interactions are predominantly driven by the consumer, while automated physical endpoints execute responsive logic based on verified commands.

4.2.1. Actor Definitions

The system interacts with several distinct entities that dictate operational flow and administrative oversight. The primary defined actors include the following participants:

- Customer: The principal user initiating the self-service shopping lifecycle through digital interactions and mobile payments.

- System Admin: The authorized personnel responsible for overseeing systemic inventory levels and operational dashboard analytics.
- Exit Gate (NodeMCU): The physical hardware endpoint that receives remote trigger commands to automatically manipulate the security barrier mechanism.

4.2.2. Functional Use Cases

The core transactions executed by these defined actors encapsulate the entire autonomous retail journey from inception to physical departure. Each functional use case guarantees strict data validation protocols prior to state transition. The primary operations include:

- Scanning the initialization QR code to logically bind the mobile application to a specific hardware basket.
- Logging physical product barcodes via the tablet scanner interface to fetch verified pricing and mass metadata.
- Validating internal weight differentials automatically using embedded load cell circuits.
- Finalizing the virtual cart summary to generate a cryptographically signed digital invoice code.
- Processing monetary settlement exclusively through the decentralized mobile terminal.
- Verifying the settled transaction at the physical boundary to trigger the solenoid unlock mechanism.

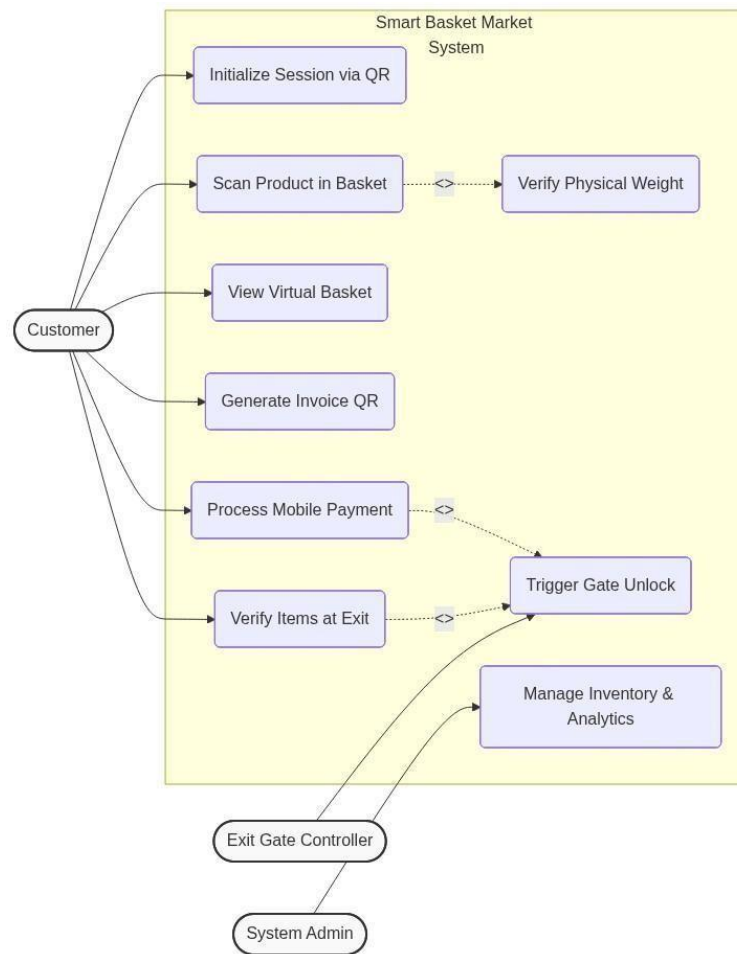


Figure 4.1: System Use Case Diagram

4.3. Sequence Diagram

The Sequence Diagram establishes the exact chronological execution order required to process multi-stage data transactions across distributed hardware nodes. This representation exposes the synchronous and asynchronous communication protocols utilized to maintain data consistency between the active frontend presentation layers and the backend persistence engine. By charting the temporal flow of digital requests, the architecture demonstrates how real-time validation loops mitigate potential structural desynchronization.

During a standard retail interaction, the temporal workflow begins with a session handshake initialized by the mobile client and concludes with a physical hardware actuation command. The intermediate operations operate within a closed processing loop, continuously crossreferencing incoming barcode strings with physical weight measurements captured by the embedded sensors. Only upon the successful culmination of the mobile payment logic does the server dispatch the final clearance signal to the boundary gate microcontroller.

Moreover, the sequence of operations places a significant emphasis on reducing the computational load on the mobile client by delegating heavy validation algorithms to the centralized backend server. The mobile application and the basket tablet function primarily as data capture nodes, responsible for capturing input and rendering server responses. This architectural decision not only enhances the overall security posture by keeping sensitive logic off consumer devices but also ensures that any updates to the pricing engine or weight tolerance algorithms can be deployed instantaneously on the backend without requiring application updates on the client side.

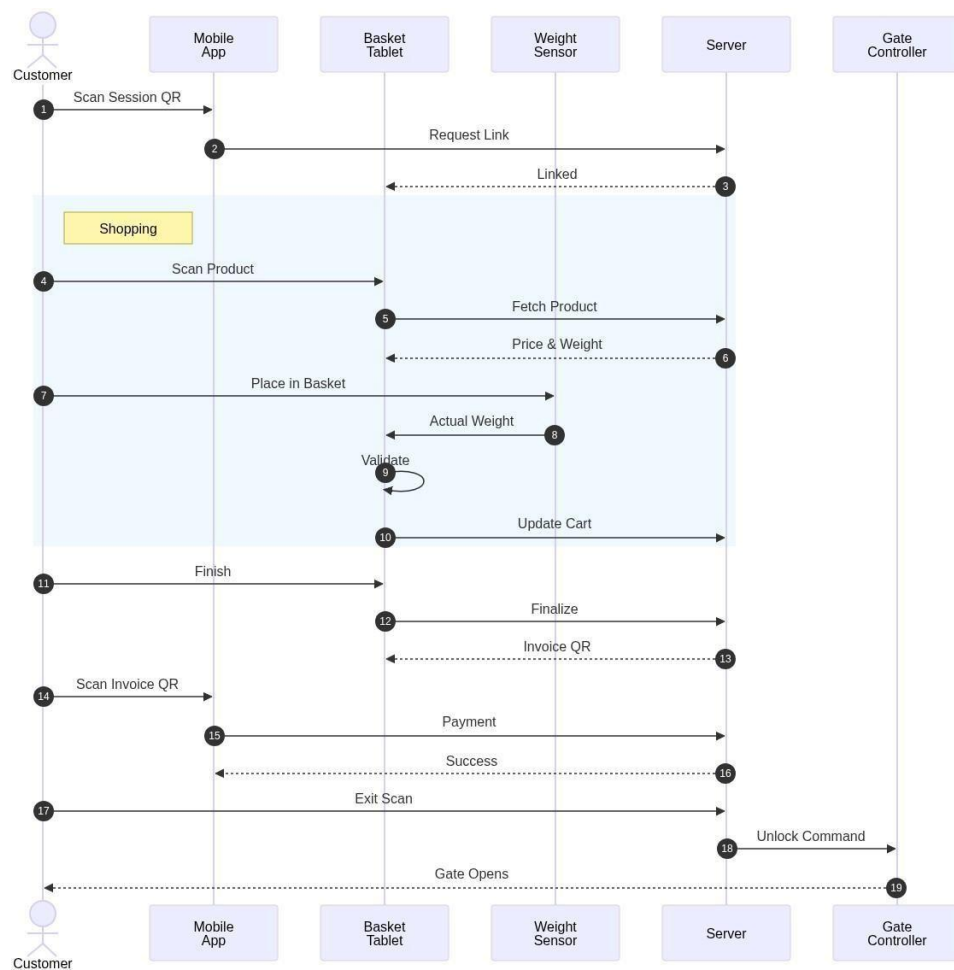


Figure 4.2: System Sequence Diagram

4.4. Activity Diagram

The Activity Diagram maps the dynamic procedural logic and conditional branching structures that govern the automated consumer journey. This behavioral model explicitly outlines the internal decision matrices utilized by the application logic to either advance the transaction or

invoke error-handling subroutines. Every physical action taken by the consumer must satisfy pre-defined systemic constraints before altering the permanent database states.

The procedural logic enforces a rigorous verification loop during the item deposition phase, comparing analog sensor conversions against authorized database metadata. Should the expected mass deviate beyond the acceptable tolerance threshold, the process halts, preventing fraudulent accumulation within the virtual cart. Upon the successful negotiation of the payment portal, the workflow transitions into an automated exit verification sequence, ensuring that physical departure is strictly coupled with successful monetary settlement.

Furthermore, the activity model illustrates the concurrent processing capabilities of the system when handling multiple operations. For instance, while the tablet interface continuously updates the visual cart rendering for the user, the background services maintain an active WebSocket listener awaiting real-time inventory synchronization signals. This approach ensures that the user experience remains highly responsive, even when the system is executing complex cryptographic validations or communicating with external banking APIs for payment authorization. It highlights the fault-tolerant nature of the architecture, where network latency or temporary sensor recalibrations do not result in a complete system crash but instead trigger graceful retry mechanisms.

Finally, the administrative and security implications of the activity flow are clearly demarcated within the final exit stages. The transition from a completed payment state to the physical unlocking of the gate involves multiple verification checkpoints designed to prevent unauthorized egress. The system logs every state change, from the initial scanning of the session QR code to the final solenoid retraction, creating a comprehensive audit trail. This detailed logging capability, embedded within the activity lifecycle, provides system administrators with the necessary telemetry to monitor operational efficiency, detect potential hardware malfunctions, and continuously optimize the autonomous checkout algorithm.

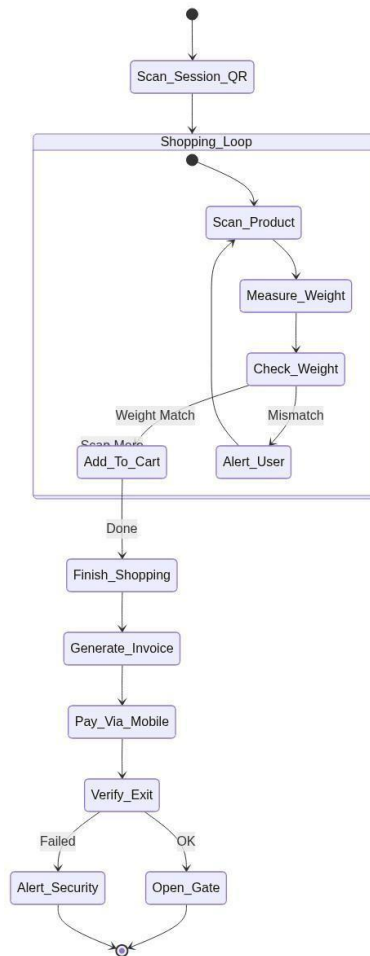


Figure 4.3: System Activity Diagram

4.5. Class Diagram

The Class Diagram provides a static, object-oriented representation of the core software structures governing the platform logic. This architectural view identifies the primary computational entities, their internal attributes, exposing methods, and relational dependencies. By encapsulating functional behavior within distinct class definitions, the framework enforces modularity, ensuring that payment processing, inventory tracking, and session management remain structurally independent.

The relational mapping connects the consumer profile directly to transient shopping sessions, which subsequently bind multiple recorded item objects into a unified digital invoice entity. The integration of the physical weight attribute within the product class definition is a critical dependency required by the automated verification loop. Furthermore, the gate controller class remains completely isolated from the cart logic, interacting solely with finalized payment states to ensure maximum operational security.

In addition to the core structural mapping, the class architecture incorporates critical design patterns that facilitate long-term maintainability and scalability. The separation of concerns is strictly enforced, where the User class handles exclusive authentication logic, while the Session class is responsible for managing the temporal lifecycle of a shopping interaction. This decoupling ensures that any future enhancements to the cryptographic hashing algorithms within the User module do not inadvertently affect the real-time operational flow of active retail sessions.

Furthermore, the CartItem and Product classes demonstrate a robust aggregational relationship that bridges the gap between static inventory data and dynamic transactional context. The Product class serves as an immutable reference blueprint containing baseline attributes such as the theoretical expected weight. Conversely, the CartItem class records the actual empirical data captured by the physical sensors during a specific session. This deliberate separation allows the system to accurately compute deviations and flag potential anomalies without altering the master inventory records, thereby preserving data integrity across all operational phases.

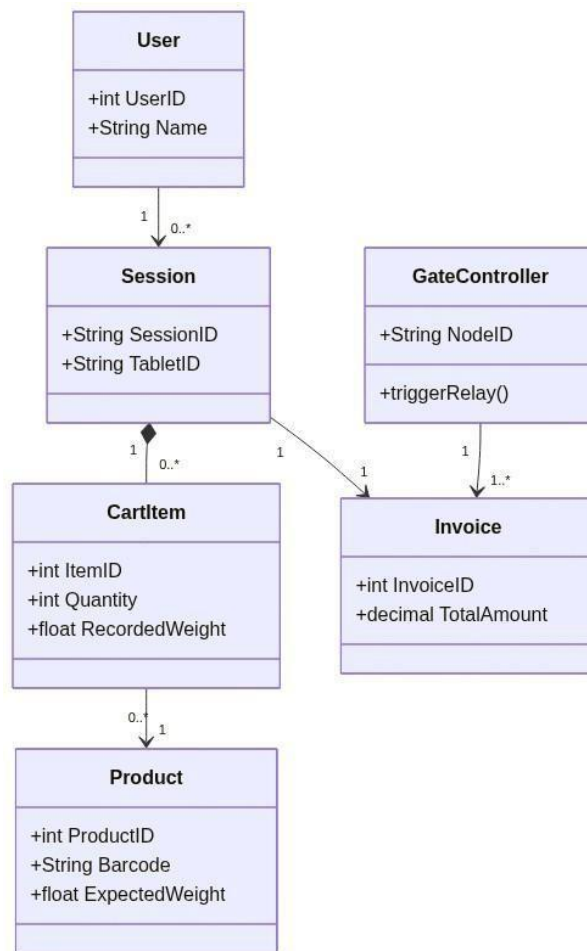


Figure 4.4: System Class Diagram

4.6. Entity Relationship Diagram (ERD)

The Entity Relationship Diagram outlines the relational database schema implemented within the persistent storage layer. This structural blueprint defines the tabular data formations, primary key constraints, and foreign key integrations necessary to guarantee absolute data normalization. Maintaining high relational integrity is paramount to supporting high-velocity concurrent queries originating from multiple distributed basket terminals.

The schema relies on five primary table structures to archive the complex interaction data generated during retail operations. The user table maintains encrypted profile credentials, while the centralized product repository serves as the definitive source for pricing and expected mass variables. All transactional events generate structured records within the sessions, invoices, and invoice items tables, creating an immutable, chronologically traceable ledger suitable for both administrative analytics and system auditing.

To ensure optimal query performance during peak operational hours, the database schema integrates selective indexing strategies across all highly contested foreign keys. The SessionID and UserID fields are heavily indexed to facilitate instantaneous linkage between active consumers and their historical transactional ledgers. This indexing is particularly vital for the mobile application, which must rapidly retrieve previous invoices and payment statuses without experiencing noticeable latency.

Moreover, the schema design enforces strict referential integrity constraints to prevent orphaned records and maintain a cohesive data ecosystem. For instance, the deletion of a User record would trigger a cascaded archival process or restriction rule, ensuring that historical financial data within the Invoices table remains mathematically sound and accessible for auditing purposes. The ERD effectively maps this multi-tiered constraint network, illustrating how the persistence layer guarantees absolute reliability even in the face of unexpected application failures or network interruptions.

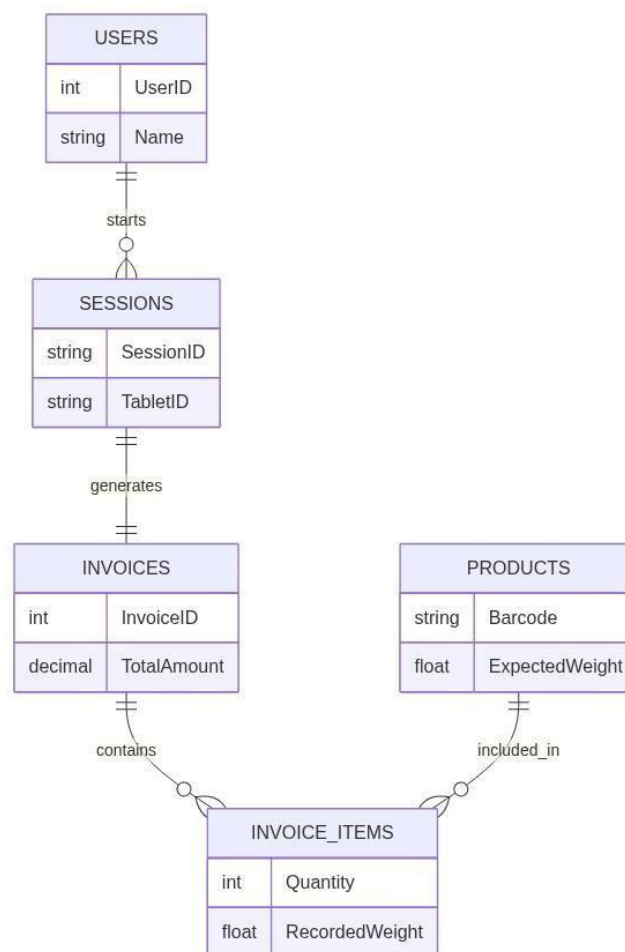


Figure 4.5: Entity Relationship Diagram

4.7. Deployment Diagram

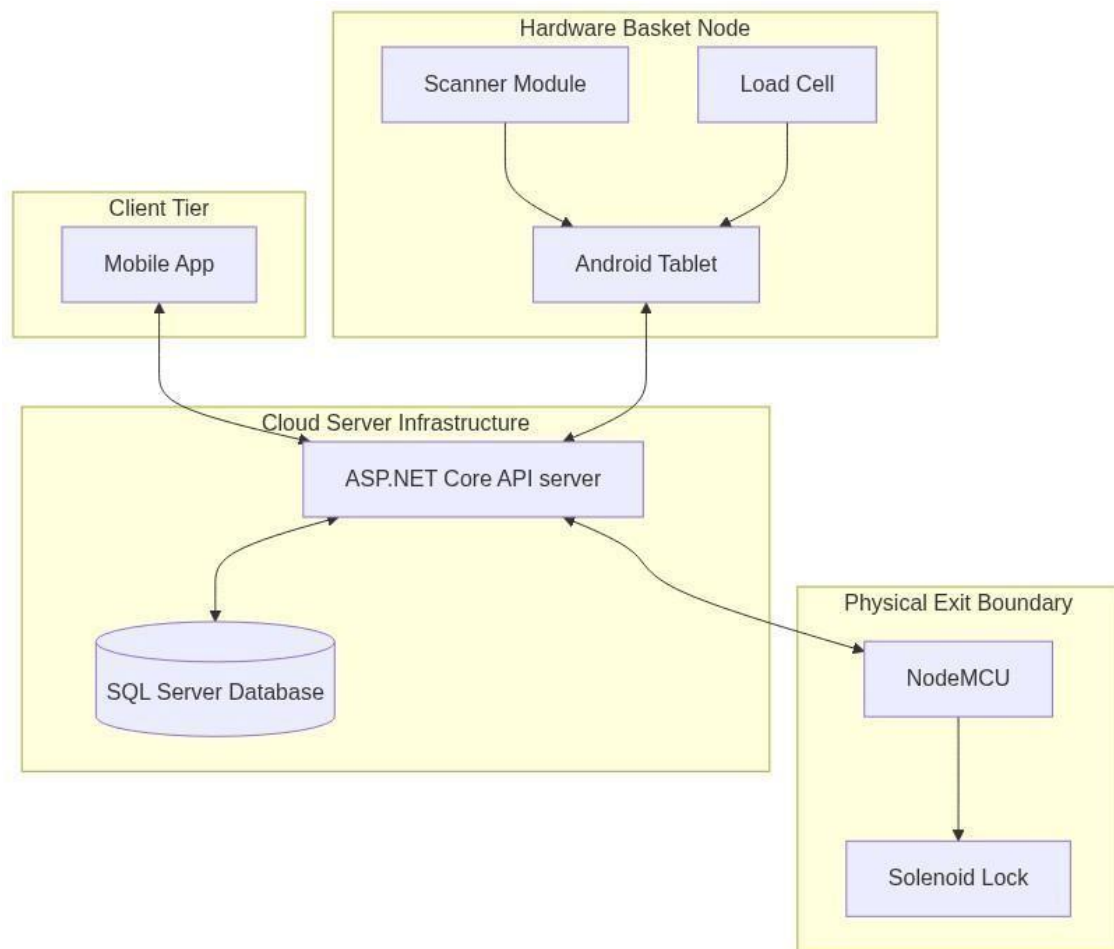


Figure 4.6: System Deployment Diagram

The Deployment Diagram details the physical distribution of hardware computational nodes and the specific communication protocols utilized to network the distributed environment. This topological model illustrates the complex infrastructure required to merge local physical subsystems with centralized cloud architectures. The physical separation of the mobile processing unit, the basket sensor module, and the exit gate assembly necessitates the implementation of diverse transmission standards.

The hardware configuration leverages wireless HTTPS protocols for standard API consumption, while employing low-latency WebSockets via SignalR for instantaneous gate actuation broadcasts. Locally on the basket assembly, analog-to-digital converters process raw load cell data through universal serial bus interfaces directly into the computing tablet. This multi-layered networking strategy ensures that high-bandwidth operations are offloaded to

capable mobile processors, while low-level physical switches receive dedicated, uninterrupted command streams.

Security protocols are inherently integrated into this deployment architecture, ensuring that all data in transit is cryptographically protected against interception. The communication channels bridging the Client Tier and the Cloud Tier utilize Transport Layer Security (TLS), encrypting sensitive payload information such as payment tokens and user credentials. Meanwhile, the localized hardware interactions between the Tablet and the Microcontroller operate within a closed physical loop, drastically reducing the attack surface against external cyber threats.

Finally, the cloud infrastructure is engineered for high availability and automatic horizontal scaling. The ASP.NET Core API servers are containerized, allowing the orchestration layer to dynamically provision additional computational resources in response to increased consumer traffic. Simultaneously, the SQL Server Database utilizes active geo-replication to safeguard against regional data center failures. This comprehensive deployment strategy guarantees that the Smart Basket Market system can sustain uninterrupted retail operations while providing an extensible foundation for future hardware integrations.

Chapter 5:

UI/UX Design

5.1 Introduction

The design of a user-facing system is as critical as its underlying technical architecture. In the context of SCAN-GO — a smart, autonomous self-checkout system embedded within a supermarket trolley — the graphical interface serves as the primary point of interaction between the shopper and the entire ecosystem of hardware and software that powers the solution. Unlike conventional retail applications designed for use in a static, controlled environment, the SCAN-GO interface must operate under uniquely demanding conditions: it is mounted on a moving cart, operated by users with varying levels of technological familiarity, used in brightly lit public spaces, and must complete entire transactional journeys — from authentication to payment to exit — with maximum speed and minimum friction.

This chapter presents a comprehensive account of the User Interface (UI) and User Experience (UX) design decisions that shape the SCAN-GO system. It begins with a foundational discussion of what UI and UX mean within this context, before examining the specific visual design language, layout strategies, interaction patterns, and user flows that were implemented. The chapter further explores the Information Architecture (IA) underpinning the application, the rationale behind key design choices related to privacy and accessibility, and a detailed screen-by-screen breakdown of the interface as it guides a shopper from initial authentication through to a seamless, queue-free store exit.

5.2 UI/UX Design

User Interface and User Experience design, while closely related, represent two distinct yet deeply intertwined disciplines. Together, they determine not only how a system looks, but how it feels and functions from the perspective of the end user. In product development, and particularly in systems designed for public-facing retail environments, a thoughtful integration of both UI and UX is essential to achieving adoption, efficiency, and trust.

UI/UX design is the multidisciplinary practice of planning, prototyping, and refining the visual layout and interactive behaviour of a digital product. It draws upon cognitive psychology, visual design theory, human factors engineering, and iterative user research to produce interfaces that are intuitive, efficient, and satisfying to use. For SCAN-GO, the UI/UX design process was guided by a core set of principles: clarity over complexity, speed of interaction, accessibility in a dynamic public environment, and unwavering attention to user privacy.

The SCAN-GO interface was designed using Figma, a professional, cloud-based design and prototyping tool widely adopted in the industry. Figma allowed for the creation of highfidelity mockups and interactive prototypes that simulate the real-world behaviour of the application. It also enabled the team to define a cohesive design system — covering colour, typography, spacing, and component styles — that ensures visual consistency across every screen of the application.

5.3 User Interface (UI)

The User Interface is the visual and interactive layer through which the user communicates with the system. It encompasses every element the user can see and touch: buttons, text labels, icons, images, colour schemes, typographic choices, and the spatial arrangement of all these components on screen. A well-designed UI reduces the mental burden placed on the user by making the path to each action immediately obvious and visually rewarding.

For SCAN-GO, the UI is rendered on a landscape-oriented tablet screen physically mounted on the shopping cart. This hardware context has a profound influence on every design decision made.

Colour Palette: The primary brand colour chosen for SCAN-GO is a vibrant, saturated emerald green (approximately #3DAA7E to #4CAF88). This colour carries strong cultural associations

with action, confirmation, commerce, and trust — all of which are entirely appropriate for a checkout application. It is used consistently across interactive elements such as primary action buttons, progress indicators, success states, and highlighted product information. The background palette transitions between a pure white (#FFFFFF) for information-dense areas such as the cart panel and payment screens, and a rich dark charcoal

(#0D1B2A to #1A2332) for the camera viewfinder panel. This

high-contrast dark background maximises the visibility of the glowing green scanning frame and camera UI elements, even under the bright fluorescent lighting typical of supermarket environments. Text is rendered in near-black (#1A1A2E, #2D2D2D) for body content and bold dark grey for secondary labels, ensuring WCAG-compliant readability at all times.

Typography: The entire interface uses a clean, geometric sans-serif typeface that prioritises legibility at a glance. Display headings — such as the "Welcome to SCAN-GO" onboarding title and the "Payment Successful!" confirmation header — are set at large sizes (36–40pt equivalents) with bold weight to command immediate attention. Product names, prices, and cart line items use a moderately large medium-weight style (18–22pt equivalents) to remain readable from a natural standing distance. Supporting labels, metadata, and secondary information are rendered at smaller sizes but always with sufficient contrast to avoid eye strain. Critically, no body text is set below the functional readability threshold, as the interface must be usable by shoppers of all ages, including elderly users who may have reduced visual acuity.

Touch Targets and Fitts's Law: The tablet is operated by touch while the user is standing beside or pushing a moving cart, which introduces a degree of physical instability not present when using a stationary device. To accommodate this, all interactive elements in the SCANGO interface are generously sized, adhering to Fitts's Law — the ergonomic principle that the time required to move to a target is a function of the target's size and distance. Primary action buttons, such as 'Tap to Scan Product', 'Add to Cart', 'Checkout', and 'Confirm Payment', span nearly the full width of the screen and carry a minimum height of approximately 56–64dp. This large target area makes them easy to tap accurately even when in motion, with gloved hands, or when distracted by the surrounding shopping environment. Interactive elements are also spaced well apart to prevent accidental taps on adjacent controls.

Icon Design: Icons are used sparingly and purposefully throughout the interface. A shopping cart icon, a camera icon, a trash/delete icon, and a WiFi/NFC icon each carry self-evident meaning that reinforces the adjacent text label without requiring the user to memorise symbol meanings. This icon-plus-label pattern is a UX best practice that improves learnability for first-time users while maintaining speed for repeat shoppers.

5.4 User Experience (UX)

User Experience design is concerned with the holistic journey a person undertakes when engaging with a product or system. Where UI design asks 'How does this look?', UX design asks 'How does this feel, and how effectively does it serve the user's real-world goal?' A positive UX is not merely the absence of frustration; it is the presence of clarity, efficiency, and — where appropriate — even delight.

The central UX challenge of SCAN-GO is that it must compress what is traditionally a multi-step, multi-location supermarket checkout process — selecting a lane, unloading items, interacting with a cashier or self-checkout terminal, bagging goods, and processing payment — into a single, continuous, in-cart experience that requires no behavioural change in how the shopper physically picks up and places items. The goal is radical convenience: the cart itself becomes the checkout, and the exit gate becomes the final confirmation.

Cognitive Load Reduction: The SCAN-GO interface is deliberately minimalist in its information density at any given moment. Rather than presenting the user with all possible actions or data simultaneously, it surfaces only the content and controls relevant to the current step in the shopping journey. The onboarding screen shows only a face scan prompt. The main dashboard shows only the camera and the cart. The payment screen focuses the user on selecting a single method. This progressive disclosure of information prevents cognitive overload and allows the user to navigate the system through sequential, low-effort decisions rather than complex multi-choice reasoning.

Frictionless Flow: Every screen transition in SCAN-GO is designed to feel inevitable and natural. Authentication leads directly to the shopping dashboard without intermediary steps. A successful product scan surfaces a contextual overlay rather than redirecting to a new page, preserving the user's spatial context. Adding an item closes the overlay and updates the cart in real time.

Payment confirmation leads directly to the exit QR code without requiring any further user action. This 'always forward' flow philosophy eliminates dead ends, back navigation requirements, and decision paralysis.

Privacy-Centred Design: A critical UX decision in SCAN-GO is the intentional elimination of PIN pad entry on the public tablet. In a traditional self-checkout environment, users must type their card PIN on a shared, visible keypad — exposing themselves to 'shoulder surfing',

where nearby individuals observe and record PIN entry. SCAN-GO eliminates this vulnerability entirely. For NFC payments, the user simply taps their card or device against the back of the

screen — a gesture that reveals no secret information. For e-wallet payments, a dynamic QR code is generated on the tablet screen, which the user then scans with their personal smartphone. All sensitive authentication and PIN entry takes place on the user's own, private device — completely shielded from public view. This design choice is not merely a feature; it is a foundational UX principle that places user security and dignity at the centre of the checkout experience.

5.6 Difference Between UI and UX

Although the terms UI and UX are frequently used interchangeably in everyday discourse, they describe fundamentally different aspects of product design.

Understanding their distinction is important in evaluating the design rigour of any digital system, including SCAN-GO.

User Interface (UI) design is primarily concerned with the visual and interactive presentation of a system. It is, in essence, the craft of deciding what a screen looks like: the colours, the fonts, the button shapes, the spacing, the iconography, and the overall aesthetic sensibility. In the context of SCAN-GO, the UI encompasses choices such as the emerald-green primary button colour, the dark-charcoal camera panel background, the large white bold typeface used for product names, the rounded corners on card and modal components, and the placement of the cart list on the right pane of the main dashboard.

These are deliberate visual decisions that contribute to brand recognition, emotional tone, and ease of visual parsing.

User Experience (UX) design, by contrast, operates at a higher level of abstraction. It is concerned with the quality of the end-to-end journey a user undertakes — including moments that precede and follow the direct interaction with the interface itself. UX encompasses questions such as: Does the biometric login feel secure and instantaneous? Is the transition from scanning a product to reviewing the cart summary cognitively smooth? Does the user feel confident that their payment information is private? Does the exit pass QR code feel like a satisfying conclusion to the shopping journey, or does it feel like an additional bureaucratic step? In SCAN-GO, UX-level thinking produced decisions such as the privacy-preserving QR

payment modal, the real-time weight validation step that silently confirms item authenticity, and the 29-second countdown timer on the exit pass screen that gently signals the system will reset for the next user.

The simplest way to contextualise the distinction is this: the UI is what the user sees; the UX is what the user feels. In SCAN-GO, both have been treated with equal seriousness, with the recognition that an interface that is visually attractive but confusing to navigate, or one that is logically structured but visually stressful, will fail to meet the product's core objective of making supermarket shopping faster, safer, and more pleasant.

5.7 Target Audience

SCAN-GO is designed to serve two primary stakeholder groups, each with distinct goals, contexts, and technical expectations.

Supermarket Shoppers (End Users): The primary target audience for the SCAN-GO tablet interface is the general supermarket-going public. This is a highly heterogeneous group spanning all age brackets, from digitally native young adults to elderly shoppers who may have limited experience with touchscreen technology. It includes individuals of varying literacy levels, those with visual or motor impairments, and users operating under time pressure or with children in tow. The UI/UX design accounts for this diversity through large text sizes, high colour contrast, simple sequential navigation, and physical affordances such as oversized touch targets. The interface explicitly avoids jargon, minimises required reading, and provides immediate visual and textual feedback for every action. The face-scan onboarding further removes the cognitive overhead of remembering passwords or loyalty card numbers, creating a barrier-free entry point accessible to essentially all shoppers.

Supermarket Retailers (Institutional Clients): The second audience for SCAN-GO consists of supermarket operators, procurement managers, and IT administrators who adopt the system at an institutional level. While these stakeholders do not directly interact with the cart-mounted tablet in the same way as shoppers, they are the decision-makers who evaluate the system against criteria such as operational efficiency gains, fraud prevention capabilities, integration with existing point-of-sale infrastructure, and compliance with financial data security standards. The design of SCAN-GO addresses their concerns through the load cell weight verification mechanism (which provides a technical deterrent to product theft), the dynamic exit pass QR system (which integrates with smart gate hardware to close the physical checkout

loop), and the privacy-preserving payment flow (which reduces the retailer's liability for card security incidents at the point of sale).

5.8 User Flow Description

The SCAN-GO user flow is a carefully sequenced series of interactions that guides the shopper from the moment they approach the cart to the moment they exit the store. Each step in the flow has been designed to be self-explanatory, reversible where appropriate, and resilient to common user errors. The complete flow is illustrated in the User Flow Diagram below.

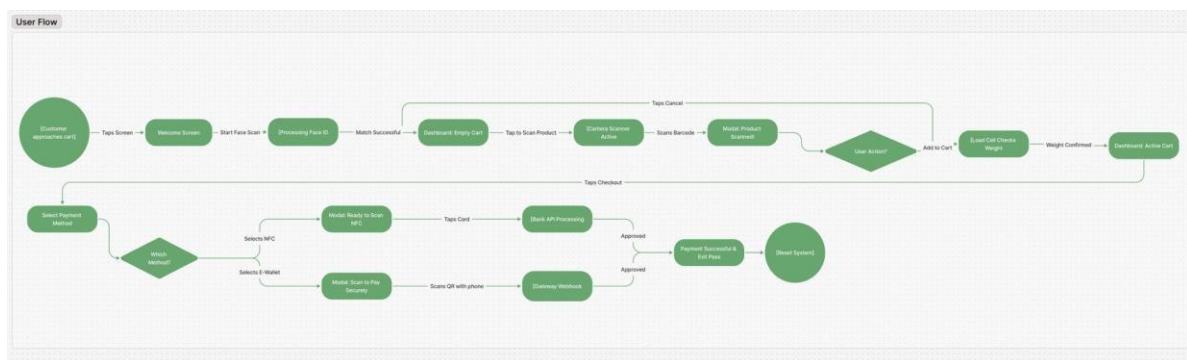


Figure 5.1 – SCAN-GO User Flow Diagram

This diagram illustrates the complete end-to-end user journey, starting from customer facial recognition login (Face ID), moving through the product scanning and automated weight verification (Load Cell) process, and concluding with the multi-channel payment integration (NFC/E-Wallet) and system reset

The flow proceeds through the following stages:

Stage 1 — Approach and Activation: The shopping journey begins when the customer physically approaches the SCAN-GO-equipped cart. The tablet screen displays the splash/onboarding screen featuring the SCAN-GO branding, the tagline "Smart Shopping Made Easy", and a clear call-to-action button: "Tap to Start Shopping". This initial screen serves as both a brand introduction and an idle-state indicator that the system is ready for a new session.

Stage 2 — Biometric Authentication: Upon tapping the activation button, the system transitions to the Biometric Login screen. The screen presents a face-scanning graphic and a "Start Face Scan" button. The user looks directly at the tablet camera. The system processes the facial scan in real time — indicated by a scanning animation with a horizontal sweep line across the face graphic and the button text changing to "Scanning...". Upon a successful match against the registered account, the system automatically proceeds to the Main Dashboard without any further input required from the user.

Stage 3 — Main Shopping Dashboard (Empty Cart): The Main Dashboard is a split-screen layout. The left pane displays the live camera viewfinder framed by a glowing green scanning rectangle. The right pane displays the active digital cart, which initially shows an empty state with the message "Your cart is empty

— Scan products to add them". A running total of \$0.00 is displayed, along with a greyed-out (inactive) "Checkout" button. The "Tap to Scan Product" button is prominently displayed at the base of the left pane.

Stage 4 — Product Scanning: When the user taps "Tap to Scan Product", the camera activates and begins actively reading barcodes. The user holds the product barcode in front of the camera. Upon a successful barcode read, a "Product Scanned!" overlay modal appears at the bottom of the screen, presenting: the product image, the product name, the price, the production date, and the expiry date. The user is then presented with two choices: "Cancel" (dismiss the modal and continue scanning) or "Add to Cart" (proceed to add the item).

Stage 5 — Weight Verification (Hardware Integration): When the user physically places the scanned item into the cart after tapping "Add to Cart", the load cell (weight sensor) embedded in the cart base registers the new weight. The system cross-references this measured weight against the expected weight stored in the product database for the scanned barcode. If the weights match within an acceptable tolerance, the item is confirmed and added to the digital cart, and the cart count and total update in real time. If there is a discrepancy

— for example, if a different or additional item was placed in the cart — the system flags an alert. This step is entirely invisible to honest shoppers but serves as a powerful antitheft mechanism.

Stage 6 — Continued Shopping and Cart Management: The user can repeat the scanning process for each item in their physical basket. The right pane of the dashboard updates in real time to reflect every added item, showing the item name, unit price, and a quantity adjustment control with plus and minus buttons. The running subtotal updates with each addition. Items

can be removed by pressing the red trash icon adjacent to each cart line. This stage loops until the user is satisfied with their selection.

Stage 7 — Checkout Initiation: When the user has finished shopping, they tap the "Checkout" button, which becomes active once at least one item is in the cart. The interface transitions to the Checkout Screen.

Stage 8 — Payment Method Selection: The Checkout Screen presents a full Order Summary on the right pane (itemised list, subtotal, tax at 15%, and grand total) alongside a left pane listing the available payment methods: (a) Tap to Pay via NFC — highlighted as the recommended option; (b) E-Wallet / Mobile Pay, supporting InstaPay, Vodafone Cash, and Orange Cash; (c) a saved Visa card ending in 4242; and (d) an option to add a new card. The user selects their preferred method.

Stage 9a — NFC Payment: If the user selects "Tap to Pay (NFC)", a modal appears instructing them to tap their card or device against the back of the tablet screen. A pulsing NFC icon provides visual feedback as the system waits for the card tap. The payment is processed via the bank API. No PIN is entered on the public screen.

Stage 9b — E-Wallet / QR Payment: If the user selects "E-Wallet / Mobile Pay", a secure QR code modal is generated. The total amount due is displayed clearly. A countdown timer (e.g., 4:59) shows the QR code expiry. The user scans this QR code with their personal smartphone and completes authorisation within their own banking or e-wallet application. The SCAN-GO system awaits a webhook callback from the payment gateway confirming approval. No sensitive data is entered on the shared tablet at any point.

Stage 10 — Payment Successful and Exit Pass: Upon payment approval, the interface transitions to the Payment Successful screen. A large green checkmark animation confirms the transaction. A digital receipt shows the amount paid, transaction ID, and timestamp. A personalised note informs the user of the time saved (e.g., "You saved 5 minutes with SCANGO today!"). On the right side of the screen, a dynamically generated Exit Pass QR code is displayed, uniquely tied to this transaction. A 29-second countdown timer indicates how long the code will remain active. The user scans this QR code at the store's smart exit gate, which is powered by a Raspberry Pi, triggering the physical barrier to open and grant exit. The system then automatically resets for the next user.

5.8 Information Architecture

Information Architecture (IA) refers to the structural organisation of a digital system's content and functionality — the blueprint that determines what information exists, where it lives, how it is grouped, and in what sequence it

becomes accessible to the user. A well-conceived IA is the invisible foundation that makes a complex system feel simple, because it ensures that users always know where they are, what they can do from their current position, and how to reach what they need.

In the case of SCAN-GO, the IA is intentionally linear rather than hierarchical. Unlike multipage web applications or feature-rich mobile apps where users might navigate freely between sections, SCAN-GO is a purpose-built transaction system with a defined start point, a defined end point, and a highly predictable sequence of interactions between them. The IA reflects this by organising the system into four primary screen zones, each with a clearly delineated set of content and functional components. The structural layout is illustrated in the Information Architecture Diagram below.

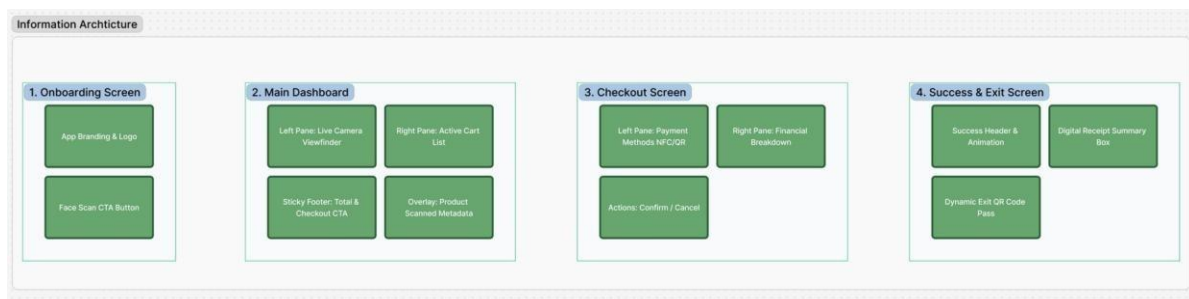


Figure 5.2 – SCAN-GO Information Architecture Diagram

This diagram details the Information Architecture of the system's user interface, mapping out the structural content and layout across four primary screens: the Onboarding Screen, Main Dashboard, Checkout Screen, and Success & Exit Screen

Zone 1 — Onboarding Screen: This zone encompasses the splash screen and the biometric login screen. Its content components are the App Branding & Logo (the SCAN-GO wordmark and icon, establishing visual identity and building trust) and the Face Scan CTA Button (the single interactive element that initiates the authentication process). By limiting Zone 1 to these

two components, the IA ensures that the entry point to the system is as low friction and clear as possible.

Zone 2 — Main Dashboard: This is the core operational zone of the application, where the majority of the shopping session takes place. It is structurally divided into three functional areas: the Left Pane, which houses the Live Camera Viewfinder and the active scanning frame; the Right Pane, which displays the Active Cart List — a real-time, scrollable register of scanned items with quantity controls and individual prices; the Sticky Footer, which contains the running Total and the Checkout CTA button, always visible regardless of

cart length; and the Overlay, which is a contextual panel that surfaces Product Scanned Metadata (name, price, production date, expiry date, product image) as a floating layer above the main dashboard without navigating away from it.

Zone 3 — Checkout Screen: This zone is activated when the user initiates the checkout process. Its left pane presents the Payment Methods (NFC, E-Wallet/QR, saved cards, and new card entry) in a scannable list format. The right pane provides the Financial Breakdown (subtotal, tax, and grand total), keeping the financial context visible throughout the payment selection process. The Actions area at the bottom of the right pane contains the Confirm Payment button and the Cancel Order & Start Over option.

Zone 4 — Success & Exit Screen: The final zone is reached upon payment confirmation. It is composed of the Success Header & Animation (the green checkmark, transaction confirmation message, and savings notification), the Digital Receipt Summary Box (amount paid, transaction ID, date and time), and the Dynamic Exit Pass QR Code — the unique, timelimited code that interfaces with the physical Raspberry Pi-powered exit gate to grant the shopper egress from the store.

5.9 User Interface Screens

The following subsections provide a detailed description of each screen and interactive state within the SCAN-GO interface, accompanied by the corresponding UI design mockups produced in Figma. Each screen has been designed to fulfil a specific role within the overall user flow while maintaining visual consistency with the overarching SCAN-GO design language.

5.10 Welcome / Onboarding Screen

The Welcome Screen is the first point of visual contact between the shopper and the SCANGO system. It is intentionally minimal: a teal-green background bearing the SCAN-GO logo (a shopping cart within a scanner frame), the brand tagline "Smart Shopping Made Easy", and a single, high-contrast call-to-action button reading "Tap to Start Shopping". The choice of a rich, single-colour background rather than a busy graphic design is deliberate — it ensures that

the interface is immediately legible under the variable and often harsh lighting conditions of a supermarket floor, and it signals to the approaching shopper

that this is an active, ready-to-use kiosk rather than a powered-off or out-of-order device.

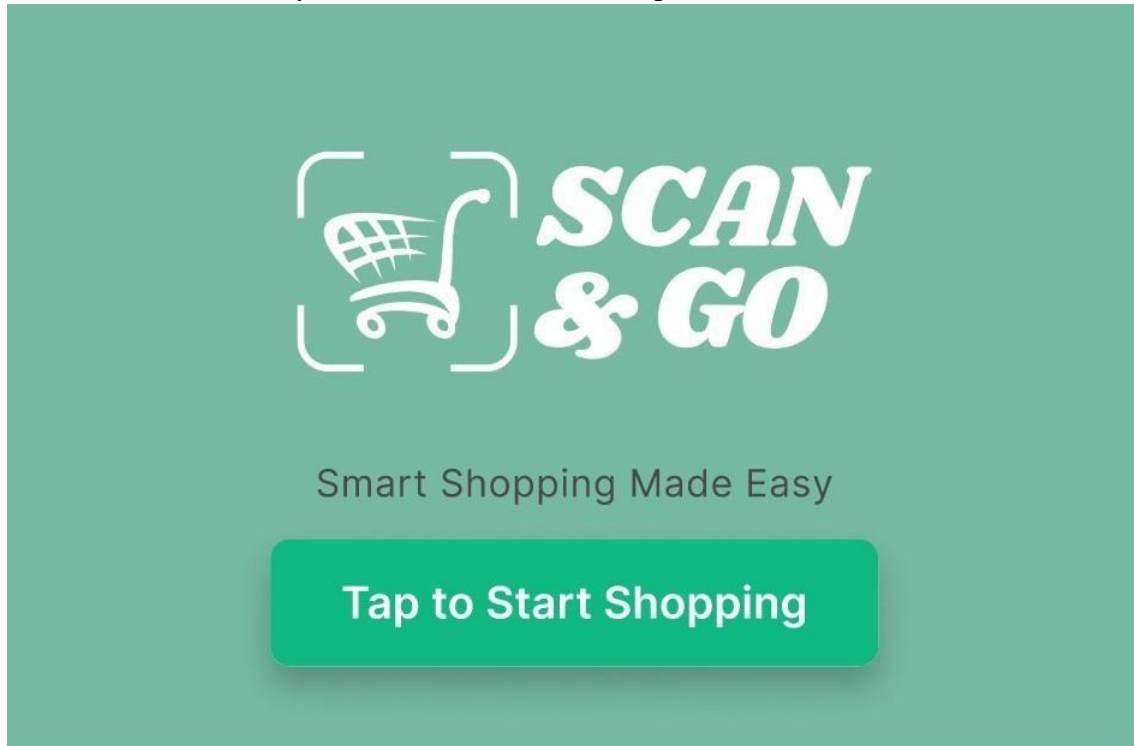


Figure 5.3 – Welcome / Onboarding Screen

This figure displays the primary user interface (splash screen) for the "SCAN & GO" system. It features the branded logo, the project slogan 'Smart Shopping Made Easy,' and a 'Tap to Start Shopping' callto-action button, designed to initiate the user flow

Upon tapping the CTA button, the system transitions to the Biometric Login screen. This screen maintains the clean white aesthetic that will persist through most of the authenticated experience. The header identifies the screen context ("SCAN-GO | Biometric Login") and the main title instructs the user: "Welcome to SCAN-GO — Look at the camera for instant, secure login." A stylised face-scanning graphic (bracket corners, eyes, nose bridge, and a smile curve in outline form) occupies the central visual space, immediately communicating the nature of the action required. The "Start Face Scan" button initiates the process.

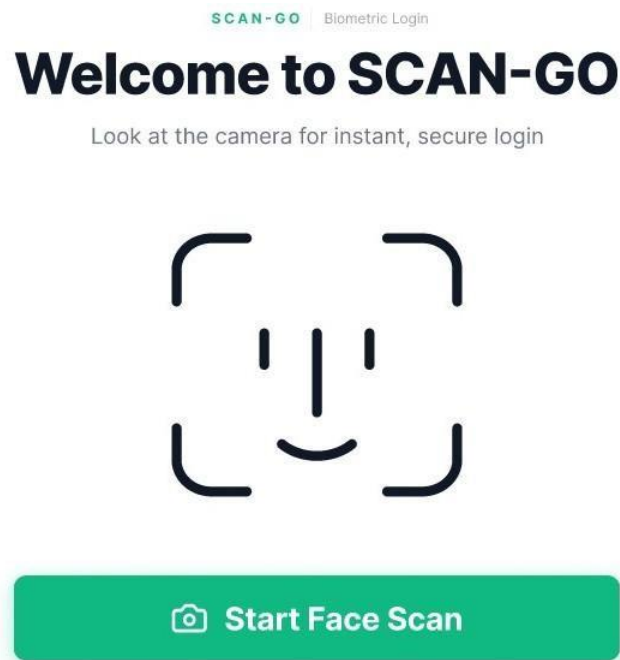


Figure 5.4 – Biometric Login Screen (Ready State)

This user interface image depicts the biometric secure login screen for the "SCAN-GO" system. It provides clear visual instructions, urging the user to "Look at the camera for instant, secure login," and features a prominent "Start Face Scan" call-to-action button, utilizing facial recognition for user authentication

During active scanning, the face graphic acquires a soft mint-green background fill, a horizontal sweep line animates across the face to indicate active sensor processing, and the button label changes to "Scanning..." — providing clear visual feedback that the system is working. A progress bar beneath the face graphic advances as the scan processes. This animated state prevents user uncertainty about whether the system has received the tap input.

SCAN-GO | Biometric Login

Welcome to SCAN-GO

Look at the camera for instant, secure login



Figure 5.5 – Biometric Login Screen (Scanning State)

This user interface image depicts the biometric secure login screen for the "SCAN-GO" system. It provides clear visual instructions, urging the user to "Look at the camera for instant, secure login," and features a prominent "Start Face Scan" call-to-action button, utilizing facial recognition for user authentication

5.11 Main Dashboard

The Main Dashboard is the operational heart of the SCAN-GO experience. It occupies the full landscape width of the tablet and is divided into two clearly delineated panes by a visual split. The left pane, rendered against the dark charcoal background, houses the live camera viewfinder framed within a glowing green rounded rectangle — the scanning target zone. A camera icon in the centre of the inactive viewfinder provides a visual affordance indicating that this area activates the camera. The "Tap to Scan Product" button, rendered in the primary SCAN-GO green, spans the base of the left pane and is the sole interactive element in this area.

The right pane of the Main Dashboard, rendered on a white background, displays the active cart. In its initial empty state, it shows a ghost cart icon, the message "Your cart is empty — Scan products to add them", and a total of \$0.00. The red "Cancel" button in the top-right corner of the right pane allows the user to abandon the session and reset the system if needed. As items are added, the right pane dynamically updates to reflect each item with its name, price, quantity controls (plus and minus buttons), and individual line-item total.

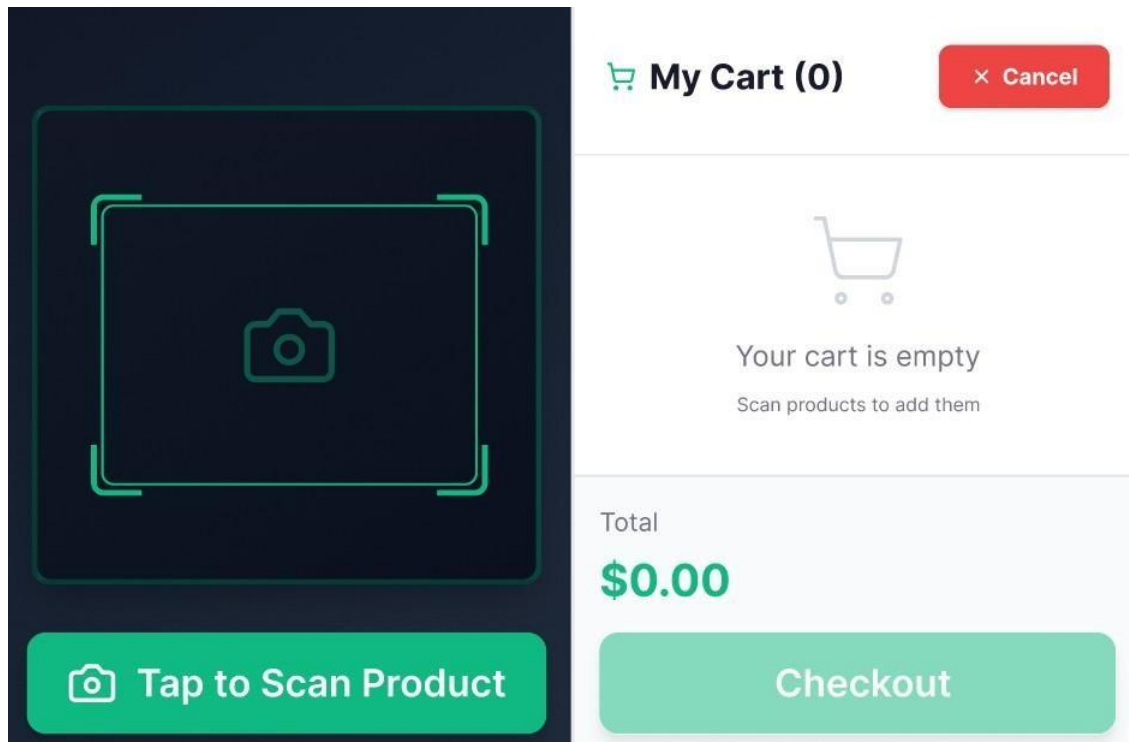


Figure 5.6 – Main Dashboard (Empty Cart State)

This user interface layout demonstrates the system's main dashboard before adding any items. The left section contains the camera scanner module with a "Tap to Scan Product" action button, while the right section displays the empty shopping cart ("My Cart (0)") state, showcasing a zeroed total and a deactivated "Checkout" button

5.12 Product Scanned Overlay

When a barcode is successfully read by the camera, the system surfaces a Product Scanned overlay at the bottom of the screen, visually anchored as a bottom sheet. This overlay displays: a product thumbnail image on the left; the product name (e.g., "Fresh Milk 1L") in bold with a green verified checkmark; the price in a prominent green typeface (e.g., "\$5.49"); a Production Date field showing the manufacture date (e.g., "Apr 25, 2026") with a calendar icon; and an Expiry Date field showing the use-by date (e.g., "May 10, 2026") with a clock icon. This expiry date information is a distinctive and consumer-beneficial feature of SCANGO, allowing shoppers to verify freshness at the point of scanning rather than discovering expired items at home.

Two action buttons appear at the right of the overlay: a red-outlined "Cancel" button to dismiss the scan result and continue, and a filled green "Add to Cart" button to confirm the addition.

This clear binary choice — with colour-coding reinforcing the desired primary action — allows the user to make a rapid decision without reading button labels in detail.

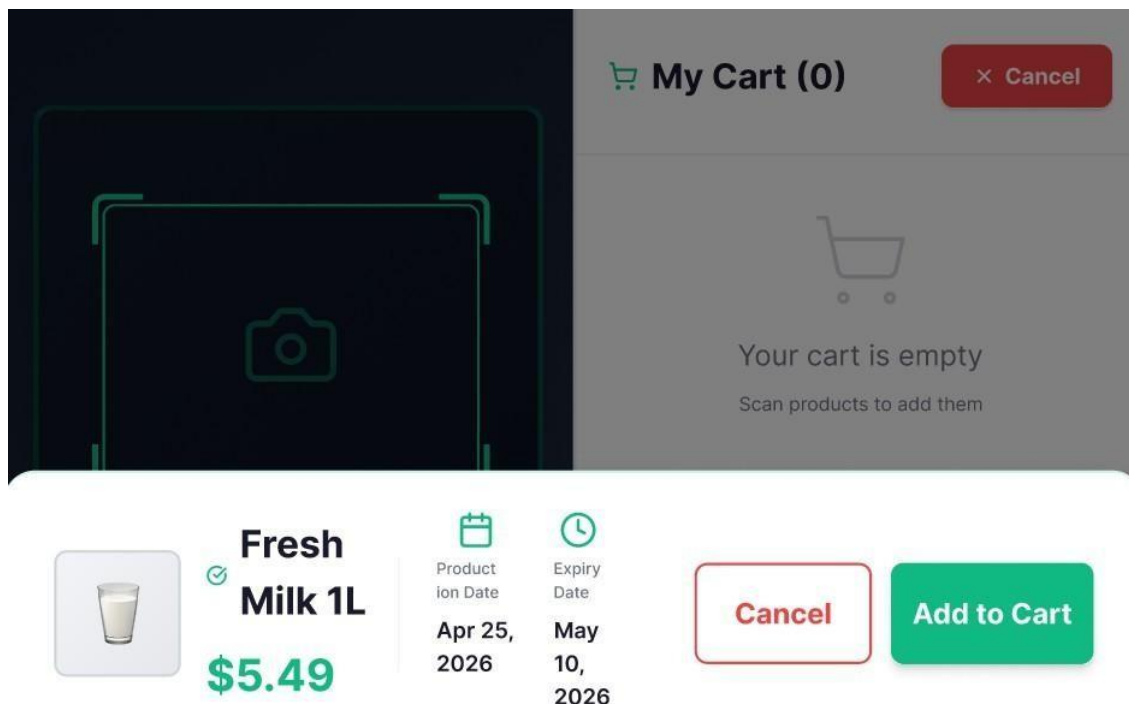


Figure 5.7 – Product Scanned Overlay Modal

This user interface layout demonstrates the system after several products have been successfully scanned and added to the cart. The right section displays a detailed list of four items: 'Fresh Milk 1L', 'Whole Wheat Bread 500g', 'Organic Cereal 300g', and 'Spring Water 500ml', each with its thumbnail, price, and green checked verification icon. The "Total" is updated to '\$15.87', and the main action button is now 'Checkout (4 items)'. The left section features a live video feed from the camera, displaying multiple stacked products with a prominent green scanner frame over one water bottle barcode. The green button below the feed is updated to show 'Scanned Barcodes (4)'.

Following confirmation, the overlay dismisses and the cart panel on the right updates immediately to reflect the newly added item. The cart item row includes the product thumbnail, name, unit price in green, quantity controls (a green minus button, the quantity count, and a green plus button), the line-item extended total, and a red trash-can icon for removal. The grand total at the base of the right pane updates accordingly, and the "Checkout" button transitions from greyed-out to fully active green.

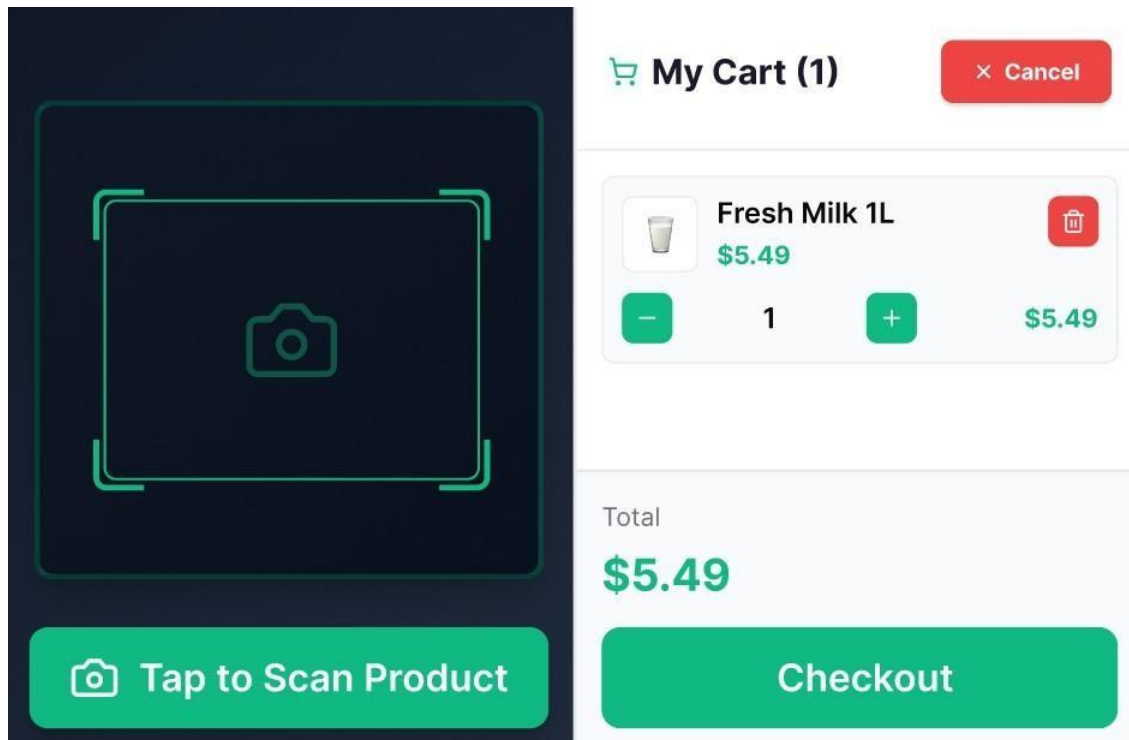


Figure 5.8 – Main Dashboard (Active Cart State, Item Added)

Figure 5.8 – Main Dashboard (Active Cart State, Item Added)

This user interface layout demonstrates the system after several products have been successfully scanned and added to the cart. The right section displays a detailed list of four items: 'Fresh Milk 1L', 'Whole Wheat Bread 500g', 'Organic Cereal 300g', and 'Spring Water 500ml', each with its thumbnail, price, and green checked verification icon. The "Total" is updated to '\$15.87', and the main action button is now 'Checkout (4 items)'. The left section features a live video feed from the camera, displaying multiple stacked products with a prominent green scanner frame over one water bottle barcode. The green button below the feed is updated to show 'Scanned Barcodes (1)'.

5.13 Select Payment Method Screen

The Payment Method Selection screen is reached when the user taps "Checkout". It presents a two-column layout. The left column lists the available payment methods in a vertically scrollable card list: "Tap to Pay (NFC)" — marked with a green "RECOMMENDED" badge and a green WiFi-signal icon on a green square background; "E-Wallet / Mobile Pay" — with an orange device icon and a descriptor listing supported platforms (InstaPay · Vodafone Cash · Orange Cash); "Visa •••• 4242" — with a blue card icon and an expiry date; and "Add New Card" — with a purple card icon and a descriptor of accepted networks.

The right column presents the Order Summary: an itemised list of purchased products with quantities and prices, a subtotal, tax (calculated at 15%), the grand total in large green text, and the green "Confirm Payment" button. A red text link at the bottom of the right column reads "Cancel Order & Start Over", providing an emergency exit from the checkout flow. The payment method rows are touch-selectable, with a green border appearing around the active selection to provide visual confirmation of the chosen method before the user taps "Confirm Payment".

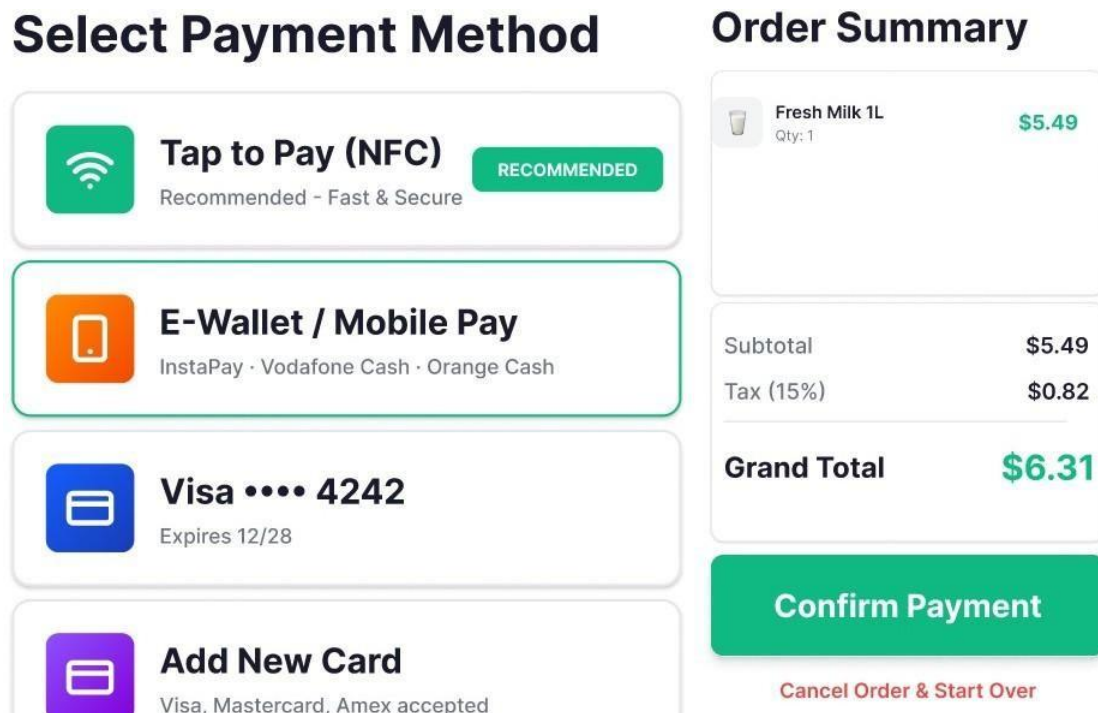


Figure 5.10 – Payment Method Selection (E-Wallet Selected)

This user interface layout presents the payment step. On the left, the user can select their preferred payment method, including the green 'Tap to Pay (NFC)' with its 'RECOMMENDED' label, the active selection 'E-Wallet / Mobile Pay' with local provider logos (InstaPay, Vodafone Cash, Orange Cash), and existing card options. On the right, the complete 'Order Summary' is detailed, showing the product breakdown (including 'Fresh Milk 1L'), a new added item (Bread \$2.99),

Subtotal, 15% Tax, and the updated 'Grand Total' of \$9.30, below which is the prominent 'Confirm Payment' action button

5.14 Secure Payment Modals

The secure payment modals are the pivotal point in the SCAN-GO UX where the privacy architecture manifests most tangibly. Two distinct modals exist depending on the selected payment method.

NFC Payment Modal: When the user selects "Tap to Pay (NFC)" and confirms, a white card modal appears over the payment screen. It features a large circular NFC/WiFi icon in mint green, the headline "Ready to Scan...", and the instruction "Please tap your card or device on the back of this screen." Three animated dots indicate that the system is actively listening for an NFC signal. A status message reads "Processing your payment...". At no point does the user enter a PIN, passcode, or any identifying information on the shared tablet screen. The payment authentication occurs through the cryptographic handshake between the card/device and the NFC reader embedded behind the tablet screen, handled entirely by the bank's API processing layer.

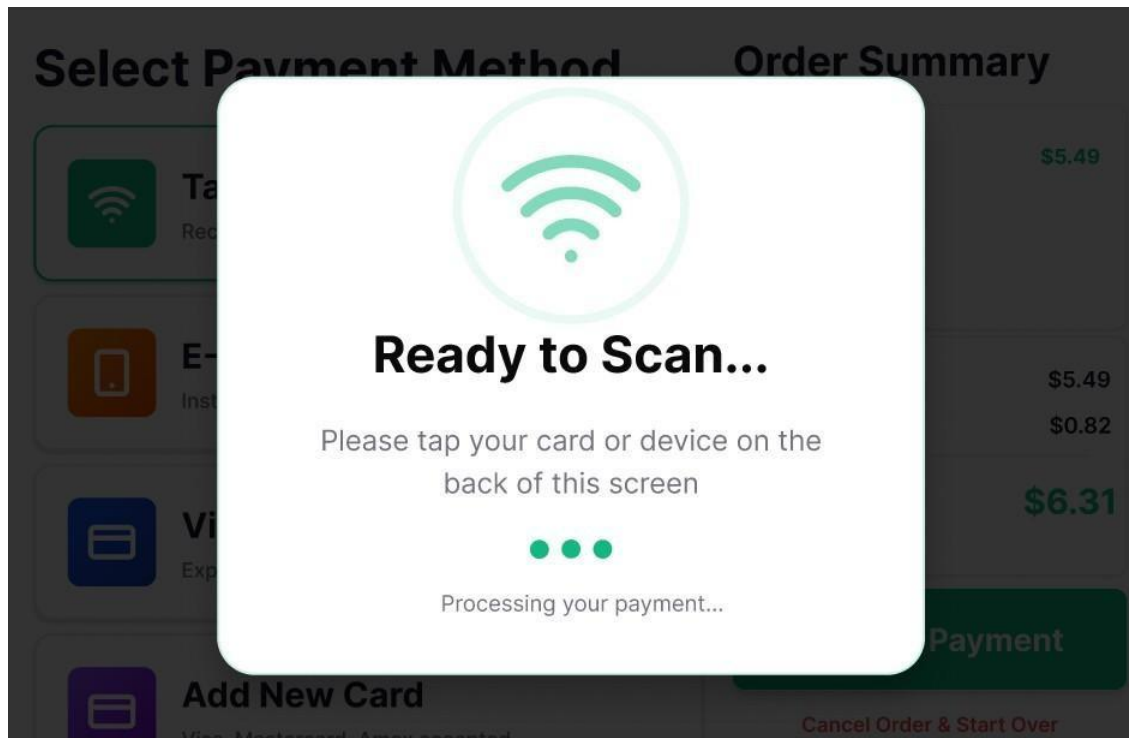
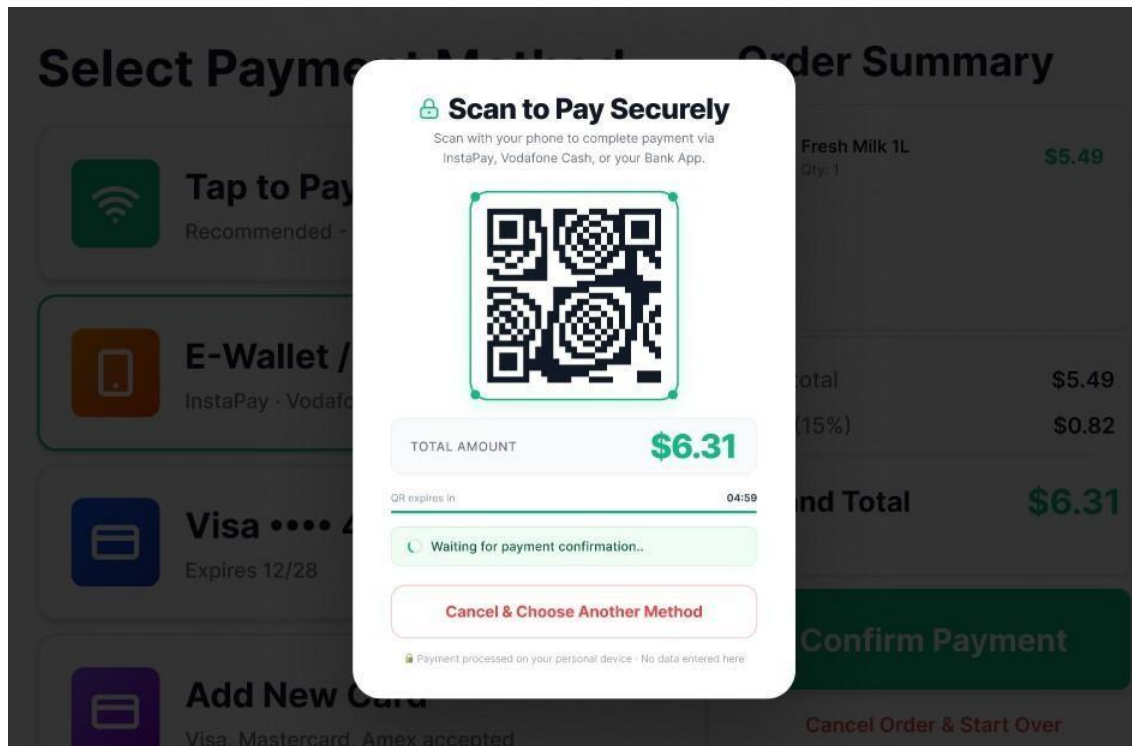


Figure 5.11 – NFC Payment Modal (Ready to Scan)

This user interface layout presents the system's final state after a successful payment transaction. The dual-pane screen shows the successful 'Tap to Pay (NFC)' method as selected on the left. The right pane details the final 'Order Summary', confirming the product, total, and tax, with all original figures preserved as green values. The main action is now the prominent 'Finished / Exit Pass' button, and a large, scannable QR code is integrated for exit verification, along with the instruction, "Please take your exit pass." All cancel and payment options are removed, finalizing the user flow.

E-Wallet / QR Payment Modal: When the user selects "E-Wallet / Mobile Pay" and confirms, a modal titled "Scan to Pay Securely" appears. It presents a dynamically generated QR code — unique to this transaction and session — that encodes the payment instruction for the specified amount. The total amount due is displayed beneath the QR code in large green text. A QR expiry countdown timer (e.g., "4:59") provides transparency about the validity

window. A status indicator reads "Waiting for payment confirmation.." with an animated spinner. A "Cancel & Choose Another Method" button allows the user to return to the payment method list. The small footer note reads "Payment processed on your personal device · No data entered here", explicitly affirming the privacy guarantee to the user. The shopper opens their own banking or e-wallet app on their personal smartphone, scans the QR code, and completes the authorisation on their private screen. The SCAN-GO system receives confirmation via a secure gateway webhook.



5.15 Payment Successful and Exit Pass Screen

The final screen of the SCAN-GO user journey is the Payment Successful and Exit Pass screen — a two-panel display that simultaneously confirms the completed transaction and enables the physical exit from the store. This screen is designed to feel celebratory and reassuring, offering the user a clear signal that their shopping session has been successfully completed and that they are free to leave.

The left panel features a large green filled circle containing a white bold checkmark — a universally recognised success symbol — followed by the headline "Payment Successful!" in bold black text. Below this, the text "Thank

you for shopping with SCAN-GO" provides a warm, personalised closing note. A summary receipt card displays the amount paid (in green), the unique transaction ID (prefixed with #SG-), and the date and time of the transaction. A mint-green highlighted banner at the bottom of the left panel celebrates the time saved by using SCAN-GO (e.g., "You saved 5 minutes with SCAN-GO today!"), leveraging the behavioural psychology of positive reinforcement to encourage repeat use of the system.

The right panel is dominated by the Exit Pass QR Code — a large, clearly rendered QR code unique to this transaction, accompanied by the transaction reference below it. The "EXIT PASS READY" label in green at the top of the right panel clearly indicates the purpose of this code. The instruction reads "Scan the QR code below at the smart exit gate to open the barrier." A green progress bar beneath the QR code counts down a 29-second active window, communicating to the user both urgency and the system's automatic reset behaviour. A prominent "Done & Exit" button provides an explicit action for users who are ready to proceed. The note "Or automatically resets in 29 seconds" manages expectations about system behaviour without requiring the user to take any further action.

At the physical level, the SCAN-GO smart exit gate — controlled by a Raspberry Pi singleboard computer equipped with a QR scanner — reads this exit pass when the user presents the tablet screen or a secondary device. Upon verification, the barrier opens, confirming that payment has been completed and allowing the shopper to exit the store. This integration of digital interface with physical hardware infrastructure represents the complete closure of the SCAN-GO transaction loop.

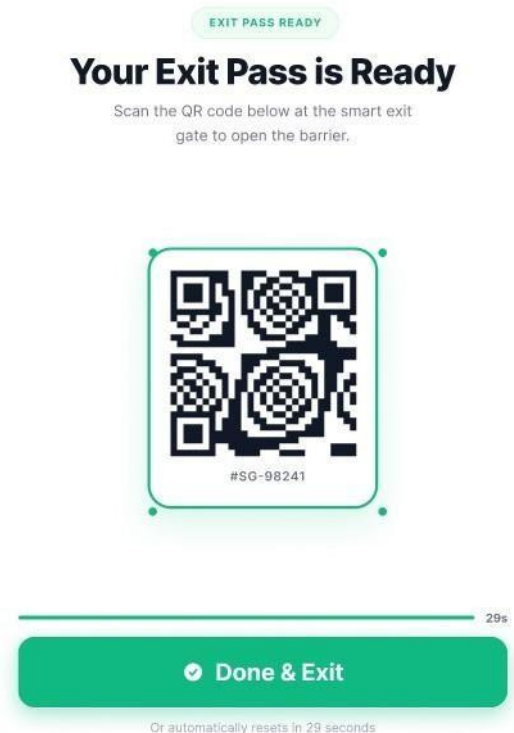
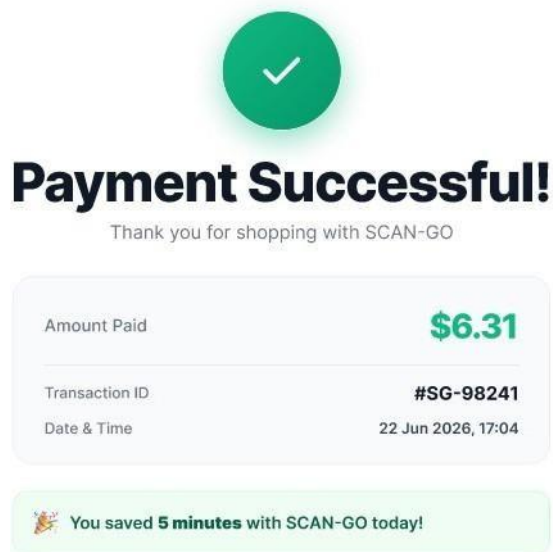


Figure 5.13 – Payment Successful and Exit Pass Screen

5.16 Security Monitor and Itemized Receipt Screens

In addition to the shopper-facing screens, SCAN-GO provides a dedicated Security Monitor interface intended for store staff and loss-prevention personnel. This screen presents a realtime view of the exit verification process for each cart, allowing an attendant to confirm — at a glance — that a shopper’s scanned items and payment are fully validated before the exit gate is released.

The header of the Security Monitor screen identifies the cart under review (“Cart #04”) alongside a live status banner confirming that a scan was just received from a specific store gate. The central panel displays a “Weight Verified & Paid” confirmation badge in green, indicating that the cart’s measured weight matches the sum of all scanned items and that payment has been successfully processed. Two supporting pill-shaped indicators reinforce this status: one confirming the matched weight in kilograms, and another confirming the absence of any anomaly between the expected and measured load. Beneath this, an Invoice Summary block presents the total number of items purchased and the total amount paid, along with the masked card number used and an “Approved” payment status tag. A “View Itemized List” control allows the attendant to drill down into the full receipt if further verification is required.

The screen concludes with two unmistakably colour-coded action buttons: a green “Open Gate” button to release the exit barrier once satisfied, and a red “Block Exit” button that allows the attendant to intervene and halt the shopper’s exit in the event of a flagged discrepancy. This staff-facing safeguard complements the shopper-facing privacy and convenience features described earlier, closing the loop between automated weight-based anti-theft detection and a human-supervised final checkpoint.

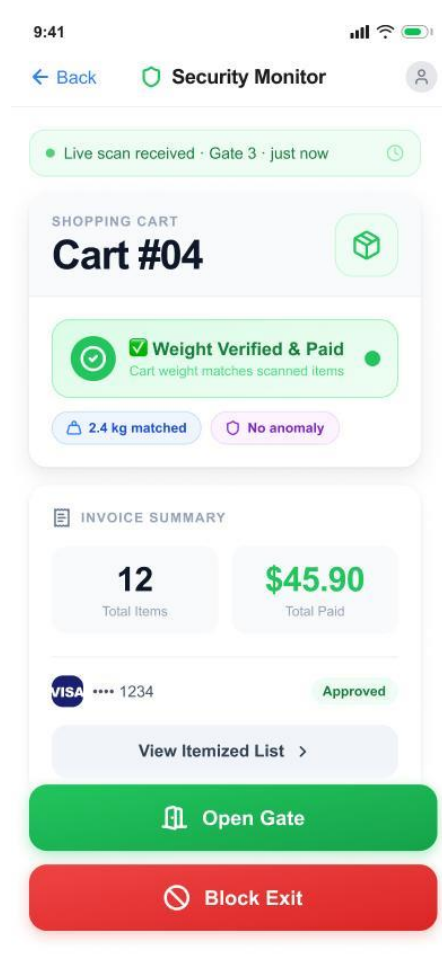


Figure 5.14 – Security Monitor Screen (Cart Verified)

Tapping “View Itemized List” on the Security Monitor screen surfaces the Itemized Receipt modal, providing full transparency into the exact contents of the cart that the weightverification system has validated. Each line item is presented with a category-coded coloured initial badge (for example, “PR” for produce, “DA” for dairy, “BA” for bakery, “ME” for meat, “BE” for beverages, and “PA” for pantry items), the product name, the quantity purchased, the category label, and the line price. This categorisation allows an attendant to quickly cross-reference the physical contents of the cart against the digital receipt during a manual spot-check.

Below the itemised list, the modal presents a financial breakdown identical in structure to the Checkout Screen’s Order Summary: a subtotal across all twelve items, a calculated tax line, and a final “Total Charged” figure rendered prominently in green. A full-width “Done” button allows the attendant to dismiss the modal and return to the main Security Monitor view. By surfacing the itemised receipt on demand rather than by default, the Information Architecture preserves the at-a-glance simplicity of the primary Security Monitor screen while still making granular detail available the moment it is needed.

CART #04	
Itemized Receipt	
PR	Organic Bananas Qty: 2 · Produce
DA	Whole Milk 2L Qty: 1 · Dairy
BA	Sourdough Bread Qty: 1 · Bakery
DA	Greek Yogurt 500g Qty: 1 · Dairy
ME	Chicken Breast 500g Qty: 1 · Meat
DA	Cheddar Cheese 200g Qty: 1 · Dairy
BE	Orange Juice 1L Qty: 1 · Beverages
PA	Spaghetti 500g Qty: 1 · Pantry
Subtotal (12 items)	
Tax (5%)	
Total Charged	
Done	

Figure 5.15 – Itemized Receipt Modal (Cart #04)

Chapter Six

Future Work

6.1 Introduction

This chapter presents the current status of the Smart Basket Market — ScanGo project, highlighting the progress achieved during the current semester and identifying the remaining tasks for the upcoming development phase. It also outlines the necessary steps required to transform the system from the design and prototyping phase into a fully integrated and deployable smart retail solution.

The project focuses on automating the customer journey within a retail environment through a smart shopping basket system. Rather than automating the entire store, the system targets only the customer-facing experience, aiming to improve the shopping process by reducing reliance on cashiers, minimizing waiting time, and eliminating congestion at checkout points.

6.2 Completed Work

During the current phase, several key achievements have been accomplished, forming the foundation for future development. These components collectively establish the architectural and design basis upon which the full system will be built.

6.2.1 Problem Analysis and Concept Definition

The inefficiencies of traditional checkout systems were analyzed, with particular focus on long waiting times and congestion resulting from full dependence on cashiers. Based on this analysis, the Smart Basket Market concept was proposed, enabling customers to scan products independently as they place them in the basket, thereby streamlining the entire checkout process without requiring dedicated cashier intervention.

6.2.2 System Requirements Analysis

Functional and non-functional requirements were defined in Chapter 3 and are briefly referenced here to maintain consistency throughout the system design. These requirements serve as the governing constraints for all subsequent development decisions.

6.2.3 System Design

An initial system design was developed to clarify user interactions and define the core components of the system. The design artifacts produced include:

- Workflow design illustrating the full customer journey
- Use Case Diagrams defining system actors and interactions
- Sequence Diagrams depicting runtime communication between components
- Class Diagrams representing the system data model

6.2.4 UI/UX Design

The user interfaces for the smart basket tablet application were designed with a focus on simplicity and accessibility. The design prioritizes a user-friendly experience that allows any customer to interact with the system without prior training, while maintaining a smooth and responsive workflow throughout the shopping session. The integrated payment module within the tablet application was also designed to ensure a seamless and intuitive checkout experience directly from the basket device.

6.3 Technical Implementation (Upcoming Phase)

The next phase will focus on transforming the system design into a fully functional solution through a structured development approach that covers the complete customer journey — from entering the store to exiting through the smart gate.

6.3.1 Smart Basket Application Development (Flutter)

The smart basket application will be developed using Flutter and installed on a low-cost Android tablet or recycled smartphone attached to each basket. The application serves as the primary point of interaction for the customer during the shopping session, and incorporates the payment functionality directly within the same interface. Key features include:

- Barcode scanning using the device camera to identify products
- Adding and removing products from the active basket session
- Updating product quantities in real time
- Displaying the live invoice as items are scanned
- Generating a QR code representing the finalized invoice upon checkout

-
- Reviewing the complete invoice details before confirming payment
- Selecting a preferred payment method

Completing a secure electronic payment transaction directly on the basket tablet

A weight sensor integrated into the basket will serve as an additional verification layer, ensuring that scanned items correspond to the actual contents of the basket. This mechanism helps detect unscanned items and supports system integrity.

The payment module embedded within the basket application is designed to be scalable, with the architecture supporting the integration of additional payment methods in future iterations.

6.3.2 Database Development

A relational database using SQL Server will be designed with indexed tables to ensure efficient query performance and fast retrieval of product and transaction data. The database will store:

- Product data including name, price, and barcode identifier
- User account data and authentication records
- Transaction records and payment history

The database design will prioritize fast access, high data integrity, and efficient management to support real-time system operations across multiple concurrent sessions.

6.3.3 Model and Algorithm Integration

The system will integrate barcode recognition algorithms to ensure accurate and fast product identification during the scanning process. In addition, a basic recommendation system based on user purchase history will be incorporated to enhance the personalized shopping experience. The recommendation approach will follow a content-based filtering technique, suggesting similar products based on user preferences and previously scanned items. These intelligent components will further enhance system value and provide dashboards for visualizing aggregated results and user insights.

6.3.4 Technology Stack Justification

The system technologies were carefully selected to ensure performance, scalability, and ease of development across all components:

- Flutter is used for cross-platform development, enabling the application to run efficiently on both customer mobile devices and smart basket tablets, including the integrated payment module
SQL Server is selected for its scalability, reliability, and ability to handle structured data efficiently
- ASP.NET is used to develop robust and secure backend services that support system integration and API management
- ESP8266 is utilized for low-cost and efficient IoT integration, enabling real-time communication with the smart gate hardware

6.4 System and Hardware Integration

This phase is critical as it transforms the individually developed components into a unified, cohesive system capable of supporting the full customer journey.

6.4.1 System Integration

All system components will be connected, including linking the basket application to the central database, integrating the payment module with the invoice engine, and ensuring real-time data synchronization across all active sessions.

6.4.2 Backend Integration

The backend built with ASP.NET will be integrated with the basket tablet application — which now incorporates the payment functionality — to support the full customer journey, beginning from session initiation upon entering the store and concluding with payment confirmation at the exit gate.

6.4.3 Hardware Integration (Smart Gate)

The system will be integrated with an IoT-based gate control mechanism using the ESP8266 module to ensure secure exit verification. Upon completing the shopping session, the customer will perform a secondary scan of the purchased items at the exit gate. The smart gate will verify that the scanned items match the paid invoice before granting exit clearance, providing a reliable loss-prevention mechanism without requiring cashier involvement.

6.5 Security and Verification

A comprehensive security mechanism will be implemented to ensure safe transactions and prevent misuse of the system. The security framework encompasses the following components:

Encryption of all payment data during transmission and storage

- A verification system at the exit gate that cross-checks scanned items against the confirmed payment record
- Real-time payment status checking at the gate (Paid / Unpaid) to prevent unauthorized exit

-

6.6 Risk Analysis

The following risks have been identified as potential threats to system reliability and continuity:

- Network failure during active shopping sessions or payment transactions
- Hardware malfunction in the smart basket tablet, weight sensor, or gate system
- Payment processing failure due to gateway or connectivity issues

The following mitigation strategies have been defined to address these risks:

- Offline caching for temporary transaction and session data storage during network interruptions
- A backup verification mechanism at the exit gate to handle connectivity failures
- Retry mechanisms for failed payment transactions to ensure transaction completion

6.7 Deployment Plan

The system will be deployed using cloud-based infrastructure, such as Microsoft Azure or AWS, to ensure scalability, high availability, and geographic redundancy. Backend services will be containerized using Docker to simplify deployment, maintenance, and horizontal scaling. APIs will be hosted in a secure cloud environment to support seamless communication between the basket application and database.

6.8 Testing and Evaluation

The system will undergo comprehensive testing across multiple dimensions to ensure correctness, performance, and security before deployment:

- **Functionality Testing:** ensuring accurate barcode scanning, correct invoice generation, and reliable QR code transactions
- **Performance Testing:** evaluating system response speed, concurrent session handling, and overall stability under load

Security Testing: verifying data protection mechanisms and preventing unauthorized system access

-
- User Experience Testing: validating ease of use for customers unfamiliar with the system

Real-world retail scenarios will also be simulated to validate end-to-end system performance under conditions representative of actual deployment environments.

6.9 Proposed Implementation Timeline

The project will be implemented across four structured phases as outlined in the following table:

Phase	Description	Duration
Phase 1	Flutter application development (Smart Basket with integrated Payment)	3 weeks
Phase 2	QR invoice generation, integrated payment module, and database development	3 weeks
Phase 3	Full system integration, IoT smart gate, and hardware setup	2 weeks
Phase 4	Testing, optimization, and final deployment	2 weeks

6.10 Summary

This chapter outlined both the achievements completed during the current semester and the detailed roadmap for future development. By combining advanced software technologies with IoT hardware integration, the Smart Basket Market — ScanGo system aims to deliver a smart, efficient, and scalable shopping experience suitable for real-world retail environments.

The system's key innovation lies in automating only the customer journey: customers scan products into their smart baskets, receive a QR-coded invoice on the tablet, complete payment independently through the integrated payment module on the same basket tablet, and exit through the smart gate after a secondary verification scan. This approach eliminates traditional checkout bottlenecks while maintaining security and data integrity throughout the

6.10 Summary

This chapter outlined both the achievements completed during the current semester and the detailed roadmap for future development. By combining advanced software technologies with IoT hardware integration, the Smart Basket Market — ScanGo system aims to deliver a

smart, efficient, and scalable shopping experience suitable for real-world retail environments.

The system's key innovation lies in automating only the customer journey: customers scan products into their smart baskets, receive a QR-coded invoice on the tablet, complete payment independently through the integrated payment module on the same basket tablet, and exit through the smart gate after a secondary verification scan. This approach eliminates traditional checkout bottlenecks while maintaining security and data integrity throughout the process. The success of the upcoming phase depends on proper implementation, rigorous integration testing, and thorough evaluation to ensure reliability and performance at scale.